Akademia Finansów i Biznesu Vistula

Wydział Informatyki, Grafiki i Architektury

Kierunek studiów Informatyka

Mateusz Maciej Bugaj

Numer albumu: 63288

APLIKACJA WSPOMAGAJĄCA ANALIZĘ I OPTYMALIZACJĘ ROZGRYWEK W GRZE DOTA 2

Praca inżynierska

napisana pod kierunkiem

Promotor - mgr inż. Maria Baczyńska-Wilkowska

Warszawa, 2026

2:767290

Streszczenie

Celem niniejszej pracy było zaprojektowanie i implementacja systemu rekomendacyjnego wspomagającego podejmowanie decyzji podczas fazy draftu w grze Dota 2. Problem draftu został potraktowany jako zadanie klasyfikacji wieloklasowej, w którym na podstawie kontekstu aktualnego stanu gry przewidywany jest najbardziej prawdopodobny wybór bohatera.

W ramach pracy pozyskano i przetworzono dane historyczne z wykorzystaniem interfejsu OpenDota API, a następnie opracowano reprezentację stanu draftu w postaci wektorów cech. Do budowy modeli wykorzystano metody uczenia nadzorowanego, w szczególności regresję logistyczną jako model bazowy oraz algorytm XGBoost jako model docelowy.

Opracowany system umożliwia generowanie rekomendacji alternatywnych wyborów bohaterów na podstawie rzeczywistych danych meczowych. Uzyskane wyniki potwierdzają, że zastosowane podejście pozwala na skuteczne odwzorowanie zależności występujących w procesie draftowania i może stanowić praktyczne wsparcie decyzyjne dla graczy.

Spis treści

1. Rozdział 1 – Wprowadzenie 1
2. Rozdział 2 – Podstawy teoretyczne 3
2.1. Gry MOBA jako system decyzyjny 3
2.2. Draft w grze Dota 2 5
2.2.1. Znaczenie kolejności wyborów 5
2.2.2. Synergie i kontrstrategie 6
2.2.3. Rola „mety” w procesie draftowania 6
2.2.4. Draft jako problem decyzyjny 7
2.3. Uczenie maszynowe w grach komputerowych 8
2.3.1. Uczenie nadzorowane 9
2.3.2. Klasyfikacja wieloklasowa 9
2.3.3. Reprezentacja stanu gry jako wektor cech 10
2.3.4. Ewaluacja modeli 10
2.3.5. Ograniczenia i wyzwania 11
2.4. Modele wykorzystane w pracy 11
2.4.1. Model bazowy jako punkt odniesienia 11
2.4.2. Regresja logistyczna 12
2.4.3. Algorytm XGBoost 13
2.4.4. Porównanie podejść 13
2.4.5. Rezygnacja z metod głębokiego uczenia 14
3. Rozdział 3 – Projekt i implementacja systemu 14
3.1. Sformułowanie problemu i założenia projektowe 15
3.2. Pozyskiwanie danych i ich charakterystyka 16
3.3. Selekcja danych i kryteria filtracji 18
3.3.1. Ograniczenie do meczów o wysokim poziomie umiejętności 18

4:338816

3.3.2. Eliminacja meczów o bardzo krótkim czasie trwania 19
3.3.3. Problem nierównomiernego rozkładu klas 19
3.3.4. Konsekwencje zastosowanej filtracji....20
3.4. Przetwarzanie danych 20
3.4.1. Parsowanie danych meczowych 20
3.4.2. Konstrukcja zbioru obserwacji draftowych....21
3.4.3. Reprezentacja danych w formacie tabelarycznym 21
3.4.4. Walidacja i kontrola jakości danych 22
3.4.5. Podsumowanie etapu przetwarzania danych 23
4. Rozdział 4 – Budowa modeli i reprezentacja danych....23
4.1. Reprezentacja stanu draftu i budowa cech 24
4.1.1. Założenia reprezentacji danych 24
4.1.2. Kodowanie bohaterów jako cech binarnych....24
4.1.3. Uwzględnienie informacji kontekstowych 25
4.1.4. Finalna postać wektora cech....25
4.1.5. Ograniczenia przyjętej reprezentacji 28
4.2. Model bazowy – regresja logistyczna 28
4.2.1. Uzasadnienie wyboru modelu bazowego 28
4.2.2. Zasada działania 29
4.2.3. Zalety i ograniczenia 29
4.2.4. Implementacja w projekcie....30
4.3. Model docelowy – XGBoost 30
4.3.1. Zasada działania algorytmu gradient boosting 30
4.3.2. Dlaczego XGBoost nadaje się do tego problemu....31
4.3.3. Konfiguracja modelu 31
4.3.4. Obsługa klasyfikacji wieloklasowej 32
4.3.5. Rola modelu w systemie rekomendacyjnym....32

5:743579

4.4. Trenowanie modeli 32
4.4.1. Podział danych na zbiory treningowe i testowe 33
4.4.2. Przygotowanie danych wejściowych 33
4.4.3. Proces uczenia modelu bazowego 33
4.4.4. Proces uczenia modelu XGBoost 33
4.4.5. Problemy napotkane w trakcie trenowania 34
4.4.6. Podsumowanie etapu trenowania 35
4.5. Ewaluacja modeli i analiza wyników 36
4.5.1. Metryki oceny jakości predykcji 36
4.5.2. Podział danych na zbiory treningowe i testowe 37
4.5.3. Porównanie modeli bazowych i zaawansowanych 37
4.5.4. Analiza błędów i ograniczeń predykcji 38
4.5.5. Wnioski z procesu ewaluacji 38
5. Rozdział 5 – Implementacja aplikacji i interfejs użytkownika 39
5.1. Architektura systemu 39
5.2. Implementacja backendu 40
5.3. Endpoint rekomendacyjny 41
5.4. Integracja modelu z aplikacją 41
5.5. Implementacja interfejsu użytkownika 42
5.6. Problemy napotkane podczas implementacji 44
5.7. Podsumowanie 45
6. Rozdział 6 – Podsumowanie i kierunki dalszego rozwoju 46
6.1. Ograniczenia zaproponowanego rozwiązania 47
6.2. Kierunki dalszego rozwoju 47
6.3. Podsumowanie końcowe 48
Bibliografia 49

6:786676

1. Rozdział 1 – Wprowadzenie

Gry komputerowe z gatunku MOBA (Multiplayer Online Battle Arena) od ponad kilkunastu lat stanowią istotny element współczesnej kultury cyfrowej oraz rynku gier komputerowych. Dynamiczny rozwój tej kategorii gier doprowadził do powstania rozbudowanego środowiska e-sportowego, w którym rywalizacja przybiera sformalizowaną i profesjonalną formę. Tytuły takie jak Dota 2 czy League of Legends przyciągają miliony graczy na całym świecie, a organizowane wokół nich turnieje generują znaczące zainteresowanie zarówno wśród widzów, jak i środowisk badawczych zajmujących się analizą danych, sztuczną inteligencją oraz teorią gier.

Jednym z najbardziej rozbudowanych i złożonych przedstawicieli gatunku MOBA jest gra Dota 2, charakteryzująca się wysokim poziomem złożoności mechanik rozgrywki oraz znaczącym wpływem decyzji podejmowanych przez graczy na ostateczny wynik meczu. Gra ta wymaga od uczestników nie tylko wysokich umiejętności manualnych, lecz także zdolności analitycznych, strategicznego myślenia oraz przewidywania działań przeciwnika.

Rozgrywka w Dota 2 opiera się na rywalizacji dwóch pięcioosobowych drużyn, których celem jest zniszczenie głównej struktury bazy przeciwnika. Każdy z graczy kontroluje jednego bohatera posiadającego unikalny zestaw umiejętności, ról oraz cech wpływających na jego funkcjonowanie w drużynie. Skuteczność zespołu zależy nie tylko od indywidualnych umiejętności graczy, lecz także od ich zdolności do współpracy oraz doboru odpowiedniej kompozycji bohaterów.

Szczególnie istotnym elementem rozgrywki jest faza draftu[1], podczas której drużyny naprzemiennie wybierają oraz blokują dostęp do określonych bohaterów. Etap ten poprzedza właściwą rozgrywkę i ma fundamentalny wpływ na jej dalszy przebieg. Decyzje podejmowane podczas draftu determinują potencjalne strategie, możliwości synergii pomiędzy bohaterami oraz podatność zespołu na działania przeciwnika. Błędy popełnione na tym etapie mogą znacząco ograniczyć szanse drużyny na zwycięstwo, nawet przy wysokim poziomie umiejętności mechanicznych graczy.

[1] Faza draftu – etap poprzedzający rozpoczęcie meczu, w którym drużyny naprzemiennie wybierają oraz blokują bohaterów dostępnych do gry.

7:827457

Złożoność procesu draftowania wynika z konieczności jednoczesnego uwzględnienia wielu czynników, takich jak role poszczególnych bohaterów, wzajemne synergie i kontrujące mechaniki, aktualna meta gry [2], a także preferencje i doświadczenie poszczególnych zawodników. W praktyce oznacza to, że podejmowanie optymalnych decyzji w fazie draftu stanowi problem wielowymiarowy, wymagający analizy dużej liczby zmiennych oraz ich wzajemnych zależności.

W ostatnich latach obserwuje się dynamiczny rozwój metod uczenia maszynowego oraz coraz większą dostępność danych pochodzących z rozgrywek sieciowych. W połączeniu z publicznie dostępnymi interfejsami programistycznymi, takimi jak OpenDota API, umożliwia to prowadzenie szczegółowych analiz statystycznych oraz budowę modeli predykcyjnych opartych na rzeczywistych danych meczowych. Wykorzystanie takich metod pozwala na identyfikację wzorców niedostrzegalnych dla człowieka oraz na wspomaganie procesu podejmowania decyzji w złożonych środowiskach decyzyjnych.

Celem niniejszej pracy jest zaprojektowanie oraz implementacja systemu wykorzystującego metody uczenia maszynowego do analizy rozegranych meczów gry Dota 2 oraz generowania rekomendacji dotyczących wyboru bohaterów w fazie draftu. Opracowane rozwiązanie działa w trybie post-game, analizując zakończone mecze na podstawie ich identyfikatorów oraz informacji kontekstowych dostępnych w danych meczowych. Na tej podstawie system generuje propozycje alternatywnych wyborów bohaterów, które potencjalnie mogłyby zwiększyć prawdopodobieństwo zwycięstwa w analogicznych sytuacjach draftowych.

W ramach pracy wykorzystano dane pochodzące z publicznego interfejsu OpenDota API, obejmujące zarówno mecze profesjonalne, jak i rozgrywki graczy o wysokim poziomie rankingowym. Dane te zostały poddane procesowi selekcji, czyszczenia oraz transformacji w celu utworzenia zbioru uczącego zawierającego informacje o kolejności wyborów, dostępnych bohaterach oraz wynikach meczów. Następnie przeprowadzono eksperymenty z wykorzystaniem wybranych algorytmów klasyfikacyjnych, w szczególności regresji logistycznej oraz modelu XGBoost, a uzyskane wyniki poddano analizie ilościowej w celu oceny skuteczności zaproponowanego podejścia.

[2] Meta (ang. *metagame*) – zbiór dominujących strategii, bohaterów i schematów rozgrywki uznawanych w danym okresie za najbardziej efektywne.

8:230986

W procesie przygotowania niniejszej pracy wykorzystano narzędzia oparte na sztucznej inteligencji jako wsparcie redakcyjne oraz techniczne. Narzędzia te były używane m.in. do konsultowania rozwiązań programistycznych, wyjaśniania błędów oraz dyskusji nad możliwymi wariantami implementacji.

Wszystkie decyzje merytoryczne, architektura systemu, implementacja kodu, dobór modeli, analiza danych oraz interpretacja wyników zostały wykonane samodzielnie przez autora. Wykorzystanie narzędzi AI miało charakter pomocniczy i nie wpłynęło na samodzielność ani oryginalność pracy.

2. Rozdział 2 – Podstawy teoretyczne

Rozdział drugi poświęcony jest omówieniu podstaw teoretycznych niezbędnych do zrozumienia dalszych części pracy. Przedstawiono w nim charakterystykę gier typu MOBA jako złożonych systemów decyzyjnych, ze szczególnym uwzględnieniem fazy draftu w grze Dota 2. Następnie omówiono wybrane zagadnienia z zakresu uczenia maszynowego, istotne z punktu widzenia budowy systemu rekomendacyjnego, w tym problem klasyfikacji wieloklasowej, reprezentację danych oraz metody oceny jakości modeli. Rozdział stanowi teoretyczne wprowadzenie do części projektowej i implementacyjnej pracy.

2.1. Gry MOBA jako system decyzyjny

Gry komputerowe z gatunku MOBA (Multiplayer Online Battle Arena) stanowią przykład złożonych, wielowymiarowych systemów decyzyjnych, w których uczestnicy podejmują szereg wzajemnie powiązanych decyzji w warunkach niepełnej informacji oraz dynamicznie zmieniającego się środowiska. Każda rozgrywka składa się z sekwencji decyzji podejmowanych zarówno indywidualnie, jak i zespołowo, przy czym skutki wielu z nich ujawniają się dopiero po upływie określonego czasu.

W odróżnieniu od klasycznych problemów optymalizacyjnych, w których dostępna jest pełna informacja o stanie systemu oraz jednoznaczna funkcja celu, gry MOBA cechują się wysokim poziomem niepewności. Gracze nie posiadają pełnej wiedzy o intencjach przeciwnika, jego planowanych strategiach ani przyszłych decyzjach. Dodatkowo każdy uczestnik musi uwzględniać zachowanie pozostałych członków własnej drużyny, co prowadzi do powstania skomplikowanej sieci zależności pomiędzy podejmowanymi działaniami.

Proces decyzyjny w tego typu grach ma charakter sekwencyjny, co oznacza, że kolejne decyzje są podejmowane na podstawie aktualnego stanu rozgrywki, który sam w sobie jest wynikiem wcześniejszych wyborów. Taka struktura powoduje, że wiele decyzji ma charakter długoterminowy — ich konsekwencje mogą nie być natychmiast widoczne, lecz wpływają na możliwości dostępne w późniejszych fazach gry. Przykładem tego typu decyzji jest wybór bohatera podczas fazy draftu, który determinuje styl gry drużyny, dostępne strategie oraz potencjalne słabości zespołu.

Z perspektywy teorii gier rozgrywki MOBA można traktować jako gry wieloosobowe o niezerowej sumie wypłat, w których interesy poszczególnych graczy są częściowo sprzeczne, a częściowo zbieżne. Ostateczny wynik meczu zależy nie tylko od jakości indywidualnych decyzji, lecz także od ich kompatybilności z decyzjami innych uczestników. W praktyce oznacza to, że nawet optymalny wybór dokonany przez jednego gracza może okazać się nieskuteczny, jeśli nie zostanie odpowiednio zsynchronizowany z działaniami reszty zespołu.

Dodatkowym utrudnieniem jest ograniczenie czasowe, w jakim decyzje muszą być podejmowane. Gracze często działają pod presją czasu, co zwiększa prawdopodobieństwo popełnienia błędów poznawczych oraz stosowania heurystyk zamiast pełnej analizy sytuacji. W takich warunkach człowiek ma naturalną tendencję do upraszczania problemu decyzyjnego, co może prowadzić do suboptymalnych wyborów.

Złożoność tych mechanizmów sprawia, że gry MOBA stanowią interesujący obiekt badań dla systemów wspomagania decyzji oraz algorytmów uczenia maszynowego. Metody te pozwalają na analizę dużych zbiorów danych historycznych w celu identyfikacji wzorców, zależności oraz korelacji, które są trudne do uchwycenia przez człowieka w czasie rzeczywistym. W szczególności możliwe jest modelowanie wpływu określonych decyzji na końcowy wynik meczu oraz estymacja prawdopodobieństwa sukcesu dla różnych scenariuszy.

Zastosowanie algorytmów sztucznej inteligencji w tym kontekście nie ma na celu zastąpienia gracza, lecz wspomaganie jego procesu decyzyjnego poprzez dostarczanie rekomendacji opartych na analizie danych. Takie podejście wpisuje się w nurt tzw. systemów wspomagania decyzji (Decision Support Systems), których zadaniem jest redukcja złożoności problemu oraz poprawa jakości podejmowanych wyborów.

10:623968

2.2. Draft w grze Dota 2

Jednym z kluczowych elementów rozgrywki w grze Dota 2 jest faza draftu, podczas której drużyny naprzemiennie dokonują wyboru oraz blokowania dostępnych bohaterów. Etap ten poprzedza właściwą część rozgrywki i w znacznym stopniu determinuje jej przebieg, możliwe strategie oraz potencjalne scenariusze rozwoju meczu.

W trakcie draftu każda z drużyn podejmuje serię decyzji polegających na wyborze własnych bohaterów (pick) oraz blokowaniu wybranych postaci przeciwnikowi (ban). Decyzje te nie są podejmowane w dowolnej kolejności, lecz zgodnie z ustalonym schematem, którego celem jest zachowanie względnej równowagi pomiędzy drużynami.

Mechanizm ten sprawia, że obie strony posiadają określone okna decyzyjne, w których mogą reagować na wybory przeciwnika, przewidywać jego strategię oraz odpowiednio modyfikować własne plany. W praktyce oznacza to, że faza draftu ma charakter silnie sekwencyjny – każda kolejna decyzja jest podejmowana w oparciu o wszystkie wcześniejsze wybory.

W rezultacie możliwa przestrzeń decyzji ulega stopniowemu zawężeniu, a dostępne opcje stają się coraz bardziej ograniczone wraz z postępem fazy draftu.

2.2.1. Znaczenie kolejności wyborów

Kolejność picków i banów ma fundamentalne znaczenie dla ostatecznego kształtu kompozycji drużyny. Wczesne wybory pozwalają na zabezpieczenie kluczowych postaci, które są uniwersalne lub szczególnie silne w danej mecie. Jednocześnie ujawniają one część planowanej strategii, co umożliwia przeciwnikowi odpowiednią reakcję.

Późniejsze etapy draftu służą często do uzupełniania brakujących ról w drużynie, wzmacniania synergii pomiędzy bohaterami lub neutralizowania konkretnych zagrożeń ze strony przeciwnika. W tej fazie decyzje są podejmowane na podstawie znacznie większej liczby informacji, ale przy jednoczesnym ograniczeniu dostępnych opcji.

W konsekwencji draft w Dota 2 nie jest prostym procesem wyboru „najlepszych” bohaterów, lecz stanowi złożony problem optymalizacyjny, w którym każda decyzja jest kompromisem pomiędzy wieloma czynnikami.

2.2.2. Synergie i kontrstrategie

Istotnym elementem procesu draftowania w grze Dota 2 są relacje pomiędzy wybieranymi bohaterami, które można podzielić na synergie oraz kontrstrategie. Synergia występuje wówczas, gdy zestaw bohaterów wzajemnie wzmacnia swoje możliwości, na przykład poprzez kompatybilne umiejętności, uzupełniające się role lub wspólny styl prowadzenia walk drużynowych.

Przykładem synergii mogą być kompozycje oparte na kontroli tłumu, inicjacji walk zespołowych lub wzmocnieniach obszarowych, gdzie skuteczność pojedynczego bohatera znacząco rośnie w obecności określonych sojuszników. W takich przypadkach wartość bohatera nie wynika wyłącznie z jego indywidualnej siły, lecz z jego miejsca w szerszym kontekście drużynowym.

Z drugiej strony istotną rolę odgrywają kontrstrategie, czyli wybory bohaterów mające na celu ograniczenie skuteczności kompozycji przeciwnika. Mogą one polegać na neutralizowaniu kluczowych umiejętności, wykorzystywaniu słabości przeciwnych bohaterów lub przeciwdziałaniu określonemu stylowi gry. Kontrwybory często podejmowane są w późniejszych etapach draftu, gdy znany jest już ogólny zarys strategii przeciwnika.

Uwzględnienie synergii oraz kontrstrategii sprawia, że draft w Dota 2 staje się procesem wymagającym analizy zależności pomiędzy wieloma zmiennymi, a skuteczność wyboru danego bohatera zależy w dużej mierze od jego interakcji z pozostałymi uczestnikami rozgrywki.

2.2.3. Rola „mety” w procesie draftowania

Istotnym czynnikiem wpływającym na decyzje podejmowane podczas draftu jest tzw. meta gry. Termin ten odnosi się do zbioru dominujących strategii, stylów rozgrywki oraz bohaterów, które w danym okresie okazują się najskuteczniejsze. Meta nie jest stała — ulega ciągłym zmianom w wyniku aktualizacji gry, balansowania postaci oraz adaptacji społeczności graczy.

Zmiany w mecie powodują, że wybory, które były optymalne kilka miesięcy wcześniej, mogą stać się nieefektywne lub wręcz ryzykowne. W związku z tym gracze oraz zespoły profesjonalne muszą stale aktualizować swoją wiedzę oraz strategie.

Z perspektywy analizy danych oznacza to, że modele predykcyjne oparte na danych historycznych są wrażliwe na zmiany w dystrybucji danych. Problem ten, znany jako concept drift, stanowi istotne wyzwanie dla systemów opartych na uczeniu maszynowym, ponieważ wymaga okresowej aktualizacji modeli oraz ponownego trenowania na nowszych danych.

2.2.4. Draft jako problem decyzyjny

Biorąc pod uwagę powyższe aspekty, fazę draftu w grze Dota 2 można formalnie traktować jako problem decyzyjny o wysokiej złożoności. Charakteryzuje się on bardzo dużą liczbą możliwych stanów gry, wynikającą z kombinacji dostępnych bohaterów, sekwencyjnym charakterem podejmowanych decyzji oraz niepełną informacją, ponieważ gracze nie znają przyszłych wyborów przeciwnika.

Dodatkowym utrudnieniem są silne zależności pomiędzy poszczególnymi decyzjami draftowymi, gdzie wybór jednego bohatera może znacząco wpływać na opłacalność kolejnych wyborów, a także obecność elementów losowych wynikających z aktualnej mety gry i zachowań przeciwników. W rezultacie przestrzeń możliwych konfiguracji draftu rośnie wykładniczo wraz z kolejnymi etapami wyborów.

Przykładowy przebieg fazy draftu w profesjonalnej rozgrywce Dota 2 przedstawiono na rysunku 1, który ilustruje sekwencyjny charakter wyborów oraz naprzemienne decyzje obu drużyn. Takie problemy są trudne do rozwiązania metodami analitycznymi, ponieważ dla człowieka oznaczają konieczność stosowania uproszczeń i heurystyk, które nie zawsze prowadzą do optymalnych rezultatów. Właśnie w tym obszarze pojawia się potencjał wykorzystania metod uczenia maszynowego. Analiza dużych zbiorów danych historycznych pozwala na estymację prawdopodobieństwa sukcesu określonych wyborów w danym kontekście draftowym, co może stanowić podstawę do budowy systemów rekomendacyjnych wspomagających proces podejmowania decyzji.



Rysunek 1. Przykładowy ekran fazy draftu w grze Dota 2 z perspektywy widza w rozgrywce profesjonalnej, ilustrujący sekwencyjny przebieg wyborów i banów bohaterów (opracowanie własne).

2.3. Uczenie maszynowe w grach komputerowych

Uczenie maszynowe (ang. machine learning) stanowi jedną z kluczowych dziedzin współczesnej sztucznej inteligencji, której celem jest tworzenie modeli zdolnych do samodzielnego uczenia się zależności na podstawie danych. W odróżnieniu od klasycznych algorytmów deterministycznych, w których wszystkie reguły są jawnie zaprogramowane przez człowieka, modele uczenia maszynowego konstruują swoje wewnętrzne reprezentacje wiedzy poprzez analizę przykładów.

W kontekście gier komputerowych metody uczenia maszynowego wykorzystywane są między innymi do tworzenia sztucznej inteligencji przeciwników, analizy zachowań graczy, wspomagania procesu balansowania rozgrywki oraz budowy systemów rekomendacyjnych. Szczególnie istotnym obszarem jest wspomaganie decyzji podejmowanych przez graczy, zwłaszcza w grach o wysokim stopniu złożoności strategicznej, takich jak Dota 2. W tego typu produkcjach liczba możliwych stanów gry oraz potencjalnych decyzji jest ogromna, co sprawia, że człowiek nie jest w stanie rozważyć wszystkich scenariuszy w sposób wyczerpujący.

2.3.1. Uczenie nadzorowane

W niniejszej pracy zastosowano podejście oparte na uczeniu nadzorowanym (ang. supervised learning). Polega ono na trenowaniu modelu na zbiorze danych, w którym dla każdego przykładu wejściowego znana jest prawidłowa odpowiedź (etykieta).

Formalnie proces ten można opisać jako aproksymację funkcji:

$f: X \rightarrow Y$

gdzie:

- X – przestrzeń cech wejściowych,
- Y – zbiór możliwych etykiet wyjściowych.

W kontekście niniejszego projektu:

- cechami wejściowymi są informacje dotyczące aktualnego stanu draftu (wybrane bohaterowie, bany, kolejność picków, strona mapy itp.),
- etykietą wyjściową jest bohater, który został wybrany w danym kontekście.

Celem modelu jest nauczenie się zależności pomiędzy kontekstem draftu a wyborem bohatera na podstawie historycznych danych z rzeczywistych meczów.

2.3.2. Klasyfikacja wieloklasowa

Problem rekomendacji bohaterów w fazie draftu można formalnie zaklasyfikować jako problem klasyfikacji wieloklasowej. Oznacza to, że model musi wybierać jedną klasę spośród wielu możliwych, gdzie każda klasa odpowiada jednemu bohaterowi.

W przypadku gry Dota 2 liczba klas jest stosunkowo duża — w momencie realizacji projektu dostępnych było ponad 120 unikalnych postaci. Tak duża liczba klas znacząco zwiększa złożoność zadania, zarówno pod względem obliczeniowym, jak i statystycznym.

Dodatkowym wyzwaniem jest nierównomierny rozkład danych. Niektórzy bohaterowie są wybierani znacznie częściej niż inni, co prowadzi do problemu niezbalansowanych klas (ang. class imbalance). Modele trenowane na takich danych mają tendencję do faworyzowania popularnych klas, co może prowadzić do obniżonej jakości predykcji dla rzadziej wybieranych postaci.

W niniejszej pracy problem ten został częściowo rozwiązany poprzez filtrację bohaterów o bardzo małej liczbie wystąpień, co pozwoliło na poprawę stabilności procesu uczenia oraz jakości ewaluacji.

15:949530

2.3.3. Reprezentacja stanu gry jako wektor cech

W projektach wykorzystujących uczenie maszynowe istotnym elementem jest sposób przygotowania danych wejściowych. Modele uczące się nie operują bezpośrednio na pojęciach takich jak stan gry czy decyzja gracza, lecz na ich numerycznych reprezentacjach.

W przypadku gier komputerowych, a w szczególności gry Dota 2, stan rozgrywki ma złożoną strukturę i nie może zostać bezpośrednio zapisany w postaci pojedynczej liczby. Konieczne jest więc jego zakodowanie w formie zbioru cech, które będą zrozumiałe dla modelu i pozwolą na uchwycenie kontekstu podejmowanej decyzji.

W ramach niniejszego projektu zastosowano reprezentację binarną, w której obecność poszczególnych bohaterów w drużynie własnej, drużynie przeciwnej lub na liście banów jest kodowana jako zmienne zero-jedynkowe. Dodatkowo uwzględniono cechy kontekstowe opisujące aktualny moment draftu, w tym stronę mapy (Radiant lub Dire), numer wyboru w sekwencji draftu, liczbę dotychczas wybranych bohaterów oraz liczbę zablokowanych postaci. Takie podejście pozwala modelowi na uchwycenie zarówno składu drużyn, jak i etapu draftu, co jest kluczowe dla poprawnej interpretacji sytuacji.

2.3.4. Ewaluacja modeli

Ocena jakości modelu klasyfikacyjnego w problemach wieloklasowych wymaga zastosowania odpowiednich miar. W niniejszej pracy wykorzystano metryki typu top-k accuracy, które mierzą, czy prawidłowa klasa znajduje się wśród k najbardziej prawdopodobnych predykcji modelu.

W praktyce oznacza to, że:

- top-1 accuracy odpowiada klasycznej dokładności,
- top-5 accuracy informuje, czy właściwy bohater znalazł się wśród pięciu najlepszych propozycji modelu.

Takie podejście jest szczególnie uzasadnione w kontekście systemów rekomendacyjnych, gdzie użytkownik nie oczekuje jednej „idealnej” odpowiedzi, lecz zbioru sensownych propozycji, spośród których może dokonać wyboru.

2.3.5. Ograniczenia i wyzwania

Pomimo wielu zalet, zastosowanie uczenia maszynowego w analizie gier MOBA wiąże się z licznymi wyzwaniami. Do najistotniejszych z nich należy zmienność metagry, wysoka złożoność przestrzeni stanów decyzyjnych, niepełna informacja dostępna w trakcie podejmowania decyzji oraz silna zależność skuteczności modeli od jakości danych wejściowych.

Modele uczone na danych historycznych nie posiadają wiedzy o przyszłych zmianach balansu gry, co oznacza, że ich skuteczność może maleć wraz z upływem czasu. Z tego względu systemy tego typu wymagają okresowej aktualizacji oraz ponownego trenowania na nowszych i bardziej reprezentatywnych danych.

2.4. Modele wykorzystane w pracy

W niniejszej pracy zastosowano dwa główne podejścia modelowe: regresję logistyczną oraz algorytm XGBoost. Modele te zostały dobrane w taki sposób, aby umożliwić zarówno stworzenie punktu odniesienia w postaci prostego i interpretowalnego rozwiązania, jak i wykorzystanie bardziej zaawansowanego algorytmu zdolnego do modelowania złożonych zależności występujących w danych pochodzących z rozgrywek gry Dota 2.

Zastosowanie dwóch modeli o różnym stopniu złożoności pozwala na przeprowadzenie rzetelnego porównania jakości predykcji oraz ocenę wpływu struktury modelu na skuteczność rekomendacji w fazie draftu.

2.4.1. Model bazowy jako punkt odniesienia

Na wczesnym etapie budowy systemu predykcyjnego istotne jest posiadanie prostego punktu odniesienia, który pozwala ocenić sensowność dalszych prac. W tym celu w projekcie wykorzystano model bazowy, którego zadaniem było określenie minimalnego poziomu skuteczności możliwego do osiągnięcia przy użyciu zastosowanych danych i cech wejściowych.

Model bazowy charakteryzuje się prostą strukturą, niewielkimi wymaganiami obliczeniowymi oraz łatwością interpretacji uzyskiwanych wyników. Dzięki temu możliwe było sprawdzenie, czy informacje opisujące stan draftu rzeczywiście zawierają zależności pozwalające na przewidywanie wyborów bohaterów.

Uzyskane rezultaty modelu bazowego stanowiły punkt odniesienia dla dalszych eksperymentów z bardziej złożonymi algorytmami, umożliwiając ocenę, czy zwiększenie złożoności modelu prowadzi do realnej poprawy jakości rekomendacji.

2.4.2. Regresja logistyczna

Jednym z zastosowanych modeli bazowych jest regresja logistyczna, będąca klasycznym algorytmem uczenia maszynowego wykorzystywanym w zadaniach klasyfikacyjnych. W kontekście niniejszego projektu model ten pełni rolę prostego punktu odniesienia, pozwalającego ocenić, w jakim stopniu na podstawie dostępnych cech możliwe jest przewidywanie wyborów bohaterów w fazie draftu.

Regresja logistyczna opiera się na prostym mechanizmie łączenia informacji zawartych w cechach wejściowych i przypisywania im wag, co umożliwia oszacowanie prawdopodobieństwa wyboru danego bohatera w określonym kontekście draftowym.

W przypadku klasyfikacji wieloklasowej, z jaką mamy do czynienia w niniejszym projekcie, stosuje się uogólnienie regresji logistycznej w postaci funkcji softmax. Pozwala ona na przypisanie każdemu możliwemu wyborowi bohatera wartości prawdopodobieństwa, przy czym suma wszystkich prawdopodobieństw wynosi 1.

Do głównych zalet regresji logistycznej należy wysoka interpretowalność uzyskiwanych wyników, niewielka liczba hiperparametrów oraz szybki proces trenowania. Dodatkowo, przy zastosowaniu odpowiednich technik regularizacji, model ten wykazuje względną odporność na nadmierne dopasowanie do danych treningowych.

Jednocześnie model ten posiada istotne ograniczenia, w szczególności trudności w modelowaniu zależności nieliniowych oraz interakcji pomiędzy cechami. W kontekście fazy draftu, gdzie relacje pomiędzy bohaterami, kolejnością wyborów oraz kontekstem sytuacyjnym są silnie nieliniowe, może to prowadzić do obniżenia jakości predykcji.

Z tego względu regresja logistyczna została wykorzystana głównie jako model odniesienia, pozwalający na ocenę minimalnego poziomu skuteczności możliwego do osiągnięcia przy wykorzystaniu analizowanych danych.

18:906377

2.4.3. Algorytm XGBoost

Drugim i głównym modelem zastosowanym w pracy jest algorytm XGBoost (ang. eXtreme Gradient Boosting). Jest to metoda oparta na wzmacnianiu gradientowym drzew decyzyjnych. Algorytm ten jest szeroko stosowany w zadaniach klasyfikacyjnych ze względu na wysoką skuteczność predykcyjną i odporność na szum w danych. Idea wzmacniania polega na iteracyjnym trenowaniu kolejnych modeli, z których każdy stara się skorygować błędy popełnione przez poprzednie. W przypadku XGBoost wykorzystywane są drzewa decyzyjne o ograniczonej głębokości, co pozwala na skuteczne modelowanie złożonych, nieliniowych zależności pomiędzy cechami.

Do kluczowych zalet algorytmu XGBoost należy zdolność do modelowania złożonych, nieliniowych interakcji pomiędzy cechami, wysoka skuteczność predykcyjna oraz odporność na szum w danych. Istotną rolę odgrywa również możliwość precyzyjnej regulacji licznych hiperparametrów oraz wbudowane mechanizmy regularizacji, które ograniczają ryzyko przeuczenia modelu.

W kontekście niniejszego projektu XGBoost jest szczególnie dobrze dopasowany do charakteru problemu. Proces draftu w grze Dota 2 jest silnie kontekstowy, a wpływ pojedynczej cechy (np. obecność danego bohatera w drużynie) zależy od wielu innych czynników, takich jak skład przeciwnika, aktualna faza draftu czy liczba dostępnych opcji. Drzewa decyzyjne pozwalają na naturalne odwzorowanie tego typu zależności.

2.4.4. Porównanie podejść

Zastosowane modele różnią się istotnie pod względem złożoności oraz możliwości predykcyjnych. Regresja logistyczna reprezentuje podejście liniowe, w którym wpływ każdej cechy na wynik jest niezależny od pozostałych. XGBoost natomiast umożliwia uwzględnienie złożonych interakcji pomiędzy cechami.

Porównanie tych modeli pozwala na analizę kompromisu pomiędzy:

- interpretowalnością a skutecznością,
- prostotą a elastycznością,
- kosztem obliczeniowym a jakością predykcji.

19:982045

W praktyce systemów rekomendacyjnych często zachodzi konieczność wyboru rozwiązania, które nie tylko generuje trafne rekomendacje, ale również pozwala na ich uzasadnienie i analizę. Z tego względu zastosowanie dwóch modeli o różnej charakterystyce jest uzasadnione zarówno metodologicznie, jak i praktycznie.

2.4.5. Rezygnacja z metod głębokiego uczenia

W niniejszej pracy świadomie zrezygnowano z zastosowania metod głębokiego uczenia, takich jak sieci neuronowe czy modele sekwencyjne. Choć podejścia te osiągają bardzo dobre wyniki w wielu zadaniach, ich użycie w analizowanym problemie wiązałoby się z szeregiem ograniczeń.

Po pierwsze, skuteczne trenowanie modeli głębokiego uczenia wymaga bardzo dużych zbiorów danych. W przypadku analizy fazy draftu dostępna liczba przykładów jest ograniczona, a dodatkowo rozkład klas jest silnie niezrównoważony.

Po drugie, modele głębokie charakteryzują się niską interpretowalnością. W kontekście systemów wspomagania decyzji istotne jest nie tylko to, jaki wybór został zasugerowany, lecz również dlaczego został uznany za korzystny.

Po trzecie, koszt obliczeniowy oraz złożoność implementacyjna modeli głębokiego uczenia znacząco utrudniłyby ich integrację z aplikacją użytkową, która stanowi istotny element niniejszej pracy.

Z tych powodów zdecydowano się na zastosowanie klasycznych metod uczenia maszynowego, które przy odpowiednim przygotowaniu danych pozwalają osiągnąć satysfakcjonujące wyniki przy zachowaniu wysokiej interpretowalności i prostoty wdrożenia.

3. Rozdział 3 – Projekt i implementacja systemu

W rozdziale trzecim przedstawiono proces projektowania oraz implementacji systemu rekomendacyjnego wspomagającego decyzje podejmowane w fazie draftu gry Dota 2. Omówiono sformułowanie problemu oraz przyjęte założenia projektowe, sposób pozyskiwania i selekcji danych meczowych, a także proces ich przetwarzania i przygotowania do dalszej analizy. Rozdział koncentruje się na aspektach praktycznych związanych z budową zbioru danych oraz przygotowaniem fundamentów pod trenowanie modeli uczenia maszynowego.

20:481869

3.1.Sformułowanie problemu i założenia projektowe

Celem niniejszego projektu było opracowanie systemu rekomendacyjnego, który wspomaga graczy gry Dota 2 w podejmowaniu decyzji podczas fazy draftu, czyli etapu, na którym drużyny wybierają oraz blokują bohaterów przed rozpoczęciem właściwej rozgrywki.
W odróżnieniu od klasycznych systemów rekomendacyjnych, które proponują użytkownikom produkty, filmy czy muzykę, analizowany problem ma charakter silnie kontekstowy. Oznacza to, że trafność rekomendacji zależy nie tylko od indywidualnych preferencji gracza, lecz również od aktualnego stanu draftu, obejmującego wcześniejsze wybory własnej drużyny, decyzje podejmowane przez przeciwnika, zablokowane postacie oraz kolejność wyboru w danej fazie draftu. Problem ten można sformułować jako zadanie klasyfikacji wieloklasowej, w którym model, na podstawie wektora cech opisującego bieżący stan draftu, przewiduje najbardziej prawdopodobnego bohatera, jaki powinien zostać wybrany w danej sytuacji. Liczba możliwych klas odpowiada liczbie dostępnych bohaterów w grze, co znacząco zwiększa złożoność problemu.

Dodatkowym utrudnieniem jest fakt, że proces draftowania ma charakter sekwencyjny. Oznacza to, że każda decyzja wpływa na przestrzeń możliwych kolejnych wyborów. Model musi zatem uwzględniać nie tylko zestaw dostępnych bohaterów, lecz również moment draftu, w którym podejmowana jest decyzja.
W ramach projektu przyjęto następujące założenia:

- Tryb post-game

System działa w trybie analizy zakończonych meczów. Użytkownik podaje identyfikator meczu, a system odtwarza przebieg fazy draftu na podstawie danych historycznych. Następnie generowane są rekomendacje alternatywnych wyborów, które mogłyby zostać dokonane w danym momencie.

- Ograniczony zakres informacji

Model nie korzysta z danych niedostępnych dla gracza w trakcie draftu, takich jak przyszłe wybory przeciwnika czy wynik meczu. Oznacza to, że rekomendacje są generowane wyłącznie na podstawie informacji, które były znane w danym momencie.

- Skupienie na wysokiej jakości danych

Do trenowania modeli wykorzystano dane pochodzące z meczów profesjonalnych oraz rozgrywek o wysokim poziomie rankingowym. Celem było ograniczenie wpływu losowych i niestandardowych decyzji, charakterystycznych dla gier o niskim poziomie zaawansowania.

- Brak adaptacji do przyszłych aktualizacji gry

System bazuje na danych historycznych i nie uwzględnia dynamicznych zmian balansu gry. Oznacza to, że jego skuteczność może ulec pogorszeniu w przypadku istotnych zmian w mecie.

- Wsparcie decyzyjne, a nie automatyzacja

System nie ma na celu zastąpienia decyzji gracza, lecz jedynie ich wspomaganie. Rekomendacje należy traktować jako sugestie, a nie jako jednoznaczne instrukcje.

Sformułowany problem charakteryzuje się wysoką złożonością zarówno pod względem struktury danych, jak i liczby możliwych decyzji. Z tego względu stanowi on interesujący obszar zastosowania metod uczenia maszynowego, szczególnie w kontekście analizy danych sekwencyjnych oraz systemów rekomendacyjnych.

3.2. Pozyskiwanie danych i ich charakterystyka

Podstawą działania systemów opartych na uczeniu maszynowym są dane, na których modele są trenowane i ewaluowane. W przypadku niniejszego projektu kluczowe znaczenie miała możliwość pozyskania dużej liczby rzeczywistych danych dotyczących przebiegu meczów gry Dota 2, w szczególności informacji związanych z fazą draftu.

Do tego celu wykorzystano publicznie dostępny interfejs programistyczny OpenDota API, który umożliwia dostęp do szczegółowych statystyk meczowych, takich jak wybory i bany bohaterów, czas trwania rozgrywki, wynik meczu, poziom rankingowy graczy czy szczegółowe dane dotyczące przebiegu gry. OpenDota stanowi popularne źródło danych wykorzystywane zarówno przez społeczność graczy, jak i badaczy analizujących rozgrywki e-sportowe.

22:308263

Proces pozyskiwania danych został w pełni zautomatyzowany przy użyciu dedykowanego skryptu, którego zadaniem było cykliczne pobieranie identyfikatorów meczów oraz sprawdzanie ich dostępności w formie w pełni sparsowanej przez serwis OpenDota. Ze względu na ograniczenia API oraz opóźnienia związane z parsowaniem meczów, konieczne było zastosowanie mechanizmów kolejkowania zapytań, opóźnień czasowych oraz wielokrotnych prób pobrania tych samych danych.

W celu zapewnienia wysokiej jakości zbioru uczącego uwzględniono wyłącznie mecze o czasie trwania przekraczającym ustalony próg, zawierające kompletne informacje o fazie draftu oraz pochodzące z rozgrywek profesjonalnych lub o bardzo wysokim poziomie rankingowym. Decyzja o ograniczeniu zbioru danych do meczów o wysokim poziomie umiejętności była podyktowana chęcią redukcji wpływu losowych i niestandardowych zachowań graczy. W rozgrywkach o niskim poziomie rankingowym często obserwuje się wybory bohaterów motywowane preferencjami osobistymi, a nie przesłankami strategicznymi, co mogłoby prowadzić do zakłócenia procesu uczenia modelu.

Kolejnym istotnym aspektem była nierównomierność rozkładu klas. W danych historycznych niektórzy bohaterowie pojawiają się znacznie częściej niż inni, co prowadzi do silnej dysproporcji pomiędzy klasami. Zjawisko to jest naturalnym skutkiem istnienia aktualnej mety gry oraz preferencji profesjonalnych drużyn. Aby ograniczyć wpływ tego problemu, zastosowano minimalny próg liczby wystąpień bohatera w zbiorze danych. Postacie, które pojawiały się rzadziej niż ustalona liczba razy, zostały usunięte z procesu treningowego.

Pozyskane dane przechowywane były w formacie JSONL, gdzie każda linia odpowiadała jednemu meczowi. Następnie dane zostały przetworzone do postaci tabelarycznej i zapisane w formacie Parquet, co umożliwiło szybkie operacje wejścia–wyjścia oraz łatwą integrację z bibliotekami analitycznymi języka Python.

W wyniku opisanych procesów uzyskano zbiór danych składający się z kilku tysięcy meczów, co przełożyło się na kilkadziesiąt tysięcy obserwacji odpowiadających pojedynczym decyzjom draftowym. Każda obserwacja reprezentuje jeden moment wyboru bohatera wraz z pełnym kontekstem widocznych w danym momencie informacji.

Tak przygotowany zbiór danych stanowił podstawę dalszych etapów projektu, w tym budowy reprezentacji cech, trenowania modeli oraz ewaluacji ich skuteczności.

3.3.Selekcja danych i kryteria filtracji

Pozyskanie dużej liczby danych nie jest równoznaczne z uzyskaniem danych wysokiej jakości. W kontekście systemów opartych na uczeniu maszynowym szczególnie istotne znaczenie ma odpowiednia selekcja obserwacji, ponieważ modele uczą się bezpośrednio na podstawie dostarczonych przykładów. Dane niskiej jakości, zawierające przypadkowe lub niereprezentatywne decyzje, mogą prowadzić do pogorszenia skuteczności predykcji oraz do powstawania błędnych wzorców.

W przypadku gry Dota 2 problem ten jest szczególnie widoczny. Gra charakteryzuje się bardzo szeroką bazą użytkowników o zróżnicowanym poziomie umiejętności, stylu gry oraz podejściu do rozgrywki. W meczach o niskim poziomie rankingowym często obserwuje się decyzje podejmowane w sposób losowy, impulsywny lub motywowane indywidualnymi preferencjami, a nie przesłankami strategicznymi. Takie zachowania, choć naturalne z perspektywy rozrywki, stanowią istotne zakłócenie w kontekście budowy systemu rekomendacyjnego.

Z tego względu konieczne było wprowadzenie zestawu kryteriów filtracji, których celem było ograniczenie wpływu przypadkowych oraz niestandardowych decyzji na proces uczenia modeli.

3.3.1. Ograniczenie do meczów o wysokim poziomie umiejętności

Jednym z podstawowych kryteriów selekcji było ograniczenie zbioru danych do meczów rozgrywanych na wysokim poziomie rankingowym oraz do rozgrywek profesjonalnych. Przyjęto, że decyzje podejmowane przez doświadczonych graczy oraz zespoły profesjonalne są bardziej przemyślane, spójne oraz oparte na aktualnej wiedzy strategicznej.

W praktyce oznaczało to wykorzystanie dostępnych w OpenDota metadanych, takich jak średni poziom rankingowy uczestników meczu (np. avg_rank_tier). Mecze poniżej ustalonego progu zostały odrzucone z dalszej analizy.

Takie podejście pozwoliło na ograniczenie wpływu losowych decyzji, zwiększenie spójności obserwowanych strategii oraz uzyskanie bardziej reprezentatywnego obrazu aktualnej mety gry. Należy jednak zaznaczyć, że takie zawężenie zbioru danych wiąże się również z pewnym kosztem — zmniejszeniem liczby dostępnych przykładów oraz zawężeniem zakresu użytkowników, dla których system może być w pełni adekwatny.

3.3.2. Eliminacja meczów o bardzo krótkim czasie trwania

Kolejnym kryterium selekcji było odrzucenie meczów o bardzo krótkim czasie trwania. Tego typu rozgrywki często kończą się przedwcześnie na skutek rozłączeń graczy, celowych poddań drużyn lub problemów technicznych, przez co nie odzwierciedlają rzeczywistego przebiegu fazy draftu. W takich przypadkach faza draftu może być niekompletna, a sam mecz nie odzwierciedla rzeczywistego przebiegu rozgrywki. Włączenie tego typu obserwacji do zbioru treningowego mogłoby prowadzić do uczenia modelu na przykładach, które nie reprezentują rzeczywistych decyzji strategicznych.

W projekcie zastosowano minimalny próg długości meczu, poniżej którego obserwacje były automatycznie odrzucane. Pozwoliło to na usunięcie przypadków niestandardowych oraz zwiększenie spójności zbioru danych.

3.3.3. Problem nierównomiernego rozkładu klas

Istotnym wyzwaniem okazał się problem nierównomiernego rozkładu klas. W kontekście niniejszego projektu każda klasa odpowiada jednemu bohaterowi. W praktyce oznacza to, że liczba klas jest bardzo duża, a ich częstość występowania znacząco się różni.

Niektórzy bohaterowie pojawiają się w danych bardzo często, podczas gdy inni są wybierani sporadycznie. Zjawisko to wynika bezpośrednio z istnienia aktualnej mety gry oraz preferencji profesjonalnych drużyn.

Tak silna nierównowaga klas może prowadzić do sytuacji, w której model uczy się głównie predykcji najczęściej występujących bohaterów, ignorując rzadziej pojawiające się klasy. W konsekwencji uzyskane rekomendacje mogą być mało zróżnicowane i powtarzalne.

Aby ograniczyć ten efekt, zastosowano minimalny próg liczby wystąpień bohatera w zbiorze danych. Postacie, które pojawiały się rzadziej niż ustalona liczba razy, zostały usunięte z procesu treningowego. Zastosowanie minimalnego progu liczby wystąpień bohatera pozwoliło na zmniejszenie poziomu szumu w danych, poprawę stabilności procesu uczenia oraz zwiększenie jakości predykcji dla pozostałych klas. Decyzja ta wiąże się jednak z kompromisem pomiędzy różnorodnością a jakością danych, co zostało szczegółowo omówione w dalszej części pracy.

25:989401

3.3.4. Konsekwencje zastosowanej filtracji

Zastosowane kryteria selekcji pozwoliły na uzyskanie zbioru danych o wyższej jakości, jednak wprowadziły również pewne ograniczenia. Przede wszystkim system został dostosowany głównie do stylu gry charakterystycznego dla wysokiego poziomu rankingowego oraz rozgrywek profesjonalnych.

Oznacza to, że rekomendacje generowane przez system mogą nie być w pełni adekwatne dla graczy o niższym poziomie zaawansowania, którzy często podejmują decyzje w oparciu o inne przesłanki, takie jak osobiste preferencje czy poziom mechanicznego opanowania bohatera.

Pomimo tych ograniczeń, przyjęte podejście uznano za uzasadnione z punktu widzenia celów projektu, których głównym założeniem było stworzenie systemu wspomagającego decyzje w oparciu o możliwie najbardziej spójne i strategiczne dane.

3.4. Przetwarzanie danych

Pozyskane z interfejsu OpenDota API dane mają postać surowych plików JSON, których struktura jest złożona i niejednorodna. Każdy rekord odpowiada jednemu meczowi i zawiera zarówno informacje kluczowe dla fazy draftu, jak i dużą liczbę danych nieistotnych z punktu widzenia niniejszego projektu. Z tego względu konieczne było przeprowadzenie wieloetapowego procesu przetwarzania danych, którego celem było uzyskanie spójnego, ustrukturyzowanego zbioru obserwacji nadającego się do dalszej analizy oraz uczenia modeli.

Proces przetwarzania danych obejmował parsowanie surowych plików JSON, selekcję istotnych informacji, transformację danych do postaci tabelarycznej oraz weryfikację poprawności uzyskanych rekordów.

3.4.1. Parsowanie danych meczowych

Surowe dane pobierane z OpenDota API zapisywane były w formacie JSONL, gdzie każda linia odpowiadała jednemu meczowi. Struktura pojedynczego rekordu zawierała m.in. informacje o przebiegu rozgrywki, statystykach graczy oraz fazie draftu zapisanej w polu picks_bans.

Pole picks_bans stanowiło kluczowy element z punktu widzenia projektu, ponieważ zawierało sekwencję zdarzeń opisujących wybory (pick) oraz blokady (ban) bohaterów wraz z informacją o kolejności oraz drużynie dokonującej decyzji. W pierwszym etapie przetwarzania dane te zostały wyodrębnione z pełnej struktury meczu, a następnie uporządkowane według kolejności wystąpienia w drafcie.

Należy podkreślić, że nie wszystkie mecze zawierały kompletne informacje o fazie draftu. W przypadku brakujących lub niekompletnych danych dany rekord był odrzucany na etapie przetwarzania, co pozwoliło uniknąć wprowadzania niejednoznacznych obserwacji do zbioru uczącego.

3.4.2. Konstrukcja zbioru obserwacji draftowych

Istotną cechą analizowanego problemu jest fakt, że pojedynczy mecz nie stanowi jednej obserwacji uczącej. Zamiast tego każdy mecz generuje wiele niezależnych obserwacji odpowiadających kolejnym decyzjom podejmowanym w trakcie draftu.

Dla każdej decyzji typu „pick” tworzony był osobny rekord danych, który zawierał:

- identyfikator wybieranego bohatera (etykieta klasy),
- listę wcześniej wybranych bohaterów własnej drużyny,
- listę bohaterów wybranych przez drużynę przeciwną,
- listę zablokowanych postaci,
- informację o stronie mapy (Radiant/Dire),
- indeks kolejności wyboru w drafcie.

Takie podejście umożliwia modelowi uczenie się zależności pomiędzy kontekstem draftu a podejmowaną decyzją. W rezultacie jeden mecz generuje od kilku do kilkunastu obserwacji uczących, w zależności od liczby decyzji draftowych spełniających przyjęte kryteria.

3.4.3. Reprezentacja danych w formacie tabelarycznym

Po wyodrębnieniu pojedynczych obserwacji draftowych dane zostały przekształcone do postaci tabelarycznej, w której każdy wiersz odpowiada jednej decyzji wyboru bohatera podjętej w trakcie fazy draftu. Taka forma reprezentacji umożliwia jednoznaczne powiązanie kontekstu draftu z wybraną postacią oraz ułatwia dalsze przetwarzanie danych.

Zbiór danych zapisano w formacie Parquet, który zapewnia wysoką wydajność operacji wejścia–wyjścia, dobrą kompresję oraz efektywną pracę z dużymi wolumenami danych. Zastosowanie reprezentacji tabelarycznej umożliwiło bezpośrednią integrację z bibliotekami analitycznymi języka Python, takimi jak pandas oraz scikit-learn, a także uprościło kolejne etapy projektu, w szczególności budowę cech oraz trenowanie modeli uczenia maszynowego.

Każdy rekord w zbiorze danych zawierał zestaw cech opisujących aktualny stan draftu, takich jak identyfikator meczu, kolejność wyboru, strona mapy czy identyfikatory wcześniej wybranych bohaterów, a także etykietę klasy w postaci identyfikatora bohatera wybranego w danej decyzji. Taka struktura danych pozwalała na bezpośrednie wykorzystanie zbioru w zadaniu klasyfikacji wieloklasowej.

Struktura pojedynczej obserwacji draftowej została zaprezentowana na rysunku 2.

```
>>
     match_id  pick_order  side  picked_hero_id  ... radiant_win duration avg_rank_tier  start_time
0  8566416347           7     1             110  ...       False     1668          None  1763558528
1  8566416347           8     0              27  ...       False     1668          None  1763558528
2  8566416347          12     0             119  ...       False     1668          None  1763558528
3  8566416347          13     1              86  ...       False     1668          None  1763558528
4  8566416347          14     1              28  ...       False     1668          None  1763558528

[5 rows x 12 columns]
```

Rysunek 2. Przykładowy fragment przetworzonego zbioru danych w formacie tabelarycznym, przedstawiający pojedyncze obserwacje draftowe (opracowanie własne).

3.4.4. Walidacja i kontrola jakości danych

Ostatnim etapem przetwarzania danych była weryfikacja poprawności uzyskanego zbioru. Przeprowadzono szereg kontroli jakości, obejmujących m.in.:

- sprawdzenie obecności brakujących wartości,
- weryfikację poprawności zakresów identyfikatorów bohaterów,
- analizę liczby obserwacji generowanych z pojedynczych meczów,
- kontrolę spójności informacji o stronie mapy oraz kolejności draftu.

28:522230

Dodatkowo w procesie przetwarzania stosowano mechanizmy logowania, umożliwiające identyfikację rekordów pominiętych lub odrzuconych na poszczególnych etapach pipeline'u. Pozwoliło to na szybkie wykrywanie błędów oraz ocenę wpływu przyjętych filtrów na końcowy rozmiar zbioru danych.

Tak przygotowany zbiór danych stanowił solidną podstawę do dalszych etapów projektu, w szczególności budowy reprezentacji cech, trenowania modeli oraz przeprowadzenia eksperymentów porównawczych.

3.4.5. Podsumowanie etapu przetwarzania danych

Przeprowadzony proces przetwarzania danych umożliwił przekształcenie surowych, niejednorodnych danych meczowych w spójny zbiór obserwacji opisujących pojedyncze decyzje draftowe. Zastosowane metody parsowania, filtracji oraz walidacji pozwoliły na uzyskanie danych o wysokiej jakości, odpowiednich do dalszych etapów analizy. Przygotowany zbiór danych stanowił podstawę do budowy reprezentacji cech oraz trenowania modeli uczenia maszynowego, opisanych w kolejnych podrozdziałach pracy.

4. Rozdział 4 – Budowa modeli i reprezentacja danych

W niniejszym rozdziale opisano proces budowy modeli uczenia maszynowego wykorzystanych w projekcie oraz sposób, w jaki przetworzone dane zostały przekształcone w postać umożliwiającą ich efektywne wykorzystanie w zadaniu predykcyjnym. Szczególną uwagę poświęcono zarówno etapowi inżynierii cech, jak i doborowi algorytmów oraz ich konfiguracji.

Proces budowy modeli został podzielony na dwa etapy. W pierwszym z nich skonstruowano model bazowy o niewielkiej złożoności, który pełnił funkcję punktu odniesienia. W drugim etapie zastosowano bardziej zaawansowany algorytm, zdolny do modelowania nieliniowych zależności pomiędzy cechami. Takie podejście pozwoliło na ocenę, w jakim stopniu wzrost złożoności modelu przekłada się na poprawę jakości rekomendacji.

W dalszej części rozdziału omówiono także proces trenowania modeli, dobór hiperparametrów oraz zastosowane metody walidacji.

29:569055

4.1. Reprezentacja stanu draftu i budowa cech

Jednym z kluczowych etapów w procesie budowy systemu opartego na uczeniu maszynowym jest odpowiednia reprezentacja danych wejściowych. Surowe informacje pozyskane z interfejsu OpenDota API, takie jak listy wybranych bohaterów czy kolejność decyzji, nie mogą zostać bezpośrednio wykorzystane przez modele klasyfikacyjne. Konieczne jest ich przekształcenie do postaci numerycznej, umożliwiającej efektywne uczenie się zależności pomiędzy kontekstem draftu a podejmowaną decyzją.

W niniejszym projekcie przyjęto podejście polegające na reprezentowaniu każdej decyzji draftowej jako wektora cech opisującego aktualny stan gry z perspektywy gracza podejmującego wybór. Wektor ten zawiera zarówno informacje o aktualnym składzie drużyn, jak i dane kontekstowe dotyczące momentu draftu.

4.1.1. Założenia reprezentacji danych

Podstawowym założeniem przyjętej reprezentacji było zachowanie zgodności z rzeczywistymi warunkami rozgrywki. Oznacza to, że model otrzymuje wyłącznie informacje, które były dostępne dla gracza w danym momencie draftu. Nie uwzględniano zatem przyszłych wyborów przeciwnika ani ostatecznego wyniku meczu.

Reprezentacja danych została zaprojektowana w taki sposób, aby umożliwiać kodowanie zmiennej liczby bohaterów wybranych do danego momentu, rozróżniać decyzje podejmowane przez własną drużynę i przeciwnika, uwzględniać zablokowane postacie oraz informację o kolejności wyboru. Istotnym wymogiem była również kompatybilność z klasycznymi algorytmami uczenia maszynowego, takimi jak regresja logistyczna czy XGBoost.

Zrezygnowano z bardziej złożonych struktur reprezentacji, takich jak grafy zależności czy modele sekwencyjne, na rzecz prostszego podejścia wektorowego. Pozwoliło to na zmniejszenie złożoności obliczeniowej, uproszczenie procesu trenowania modeli oraz ułatwienie interpretacji uzyskanych wyników.

4.1.2. Kodowanie bohaterów jako cech binarnych

Podstawowym elementem wektora cech jest informacja o obecności poszczególnych bohaterów w aktualnym stanie draftu. W tym celu zastosowano kodowanie binarne (ang. one-hot encoding), polegające na przypisaniu osobnej cechy dla każdego bohatera.

Dla każdej postaci zdefiniowano trzy potencjalne typy cech: obecność bohatera w drużynie własnej (ally_ID), obecność w drużynie przeciwnika (enemy_ID) oraz fakt jego zablokowania w drafcie (ban_ID). Wartość danej cechy przyjmuje wartość 1, jeśli bohater występuje w danej kategorii, lub 0 w przeciwnym przypadku. Takie podejście umożliwia jednoznaczne rozróżnienie roli każdej postaci w aktualnym stanie draftu.

Przykładowo, jeśli drużyna wybrała bohatera o identyfikatorze 25, a przeciwnik bohatera 17, odpowiednie cechy przyjmują wartości ally_25 = 1 oraz enemy_17 = 1, podczas gdy pozostałe cechy pozostają równe zero. Rozwiązanie to jest intuicyjne, łatwe w interpretacji oraz kompatybilne z większością algorytmów klasyfikacyjnych.

4.1.3. Uwzględnienie informacji kontekstowych

Sama informacja o wybranych bohaterach nie jest wystarczająca do poprawnego modelowania procesu draftowania. Kluczowe znaczenie ma również kontekst, w jakim podejmowana jest decyzja.

W związku z tym do wektora cech wprowadzono dodatkowe informacje opisujące moment draftu. Uwzględniono stronę mapy, po której znajduje się drużyna dokonująca wyboru, numer kolejności wyboru oraz uproszczony podział draftu na fazę wczesną, środkową i późną. Dodatkowo model otrzymuje informację o liczbie bohaterów wybranych przez obie drużyny oraz liczbie zablokowanych postaci.

Cechy te pozwalają rozróżniać sytuacje, w których identyczny zestaw bohaterów pojawia się w różnych momentach draftu, co w praktyce może prowadzić do zupełnie innych decyzji strategicznych.

4.1.4. Finalna postać wektora cech

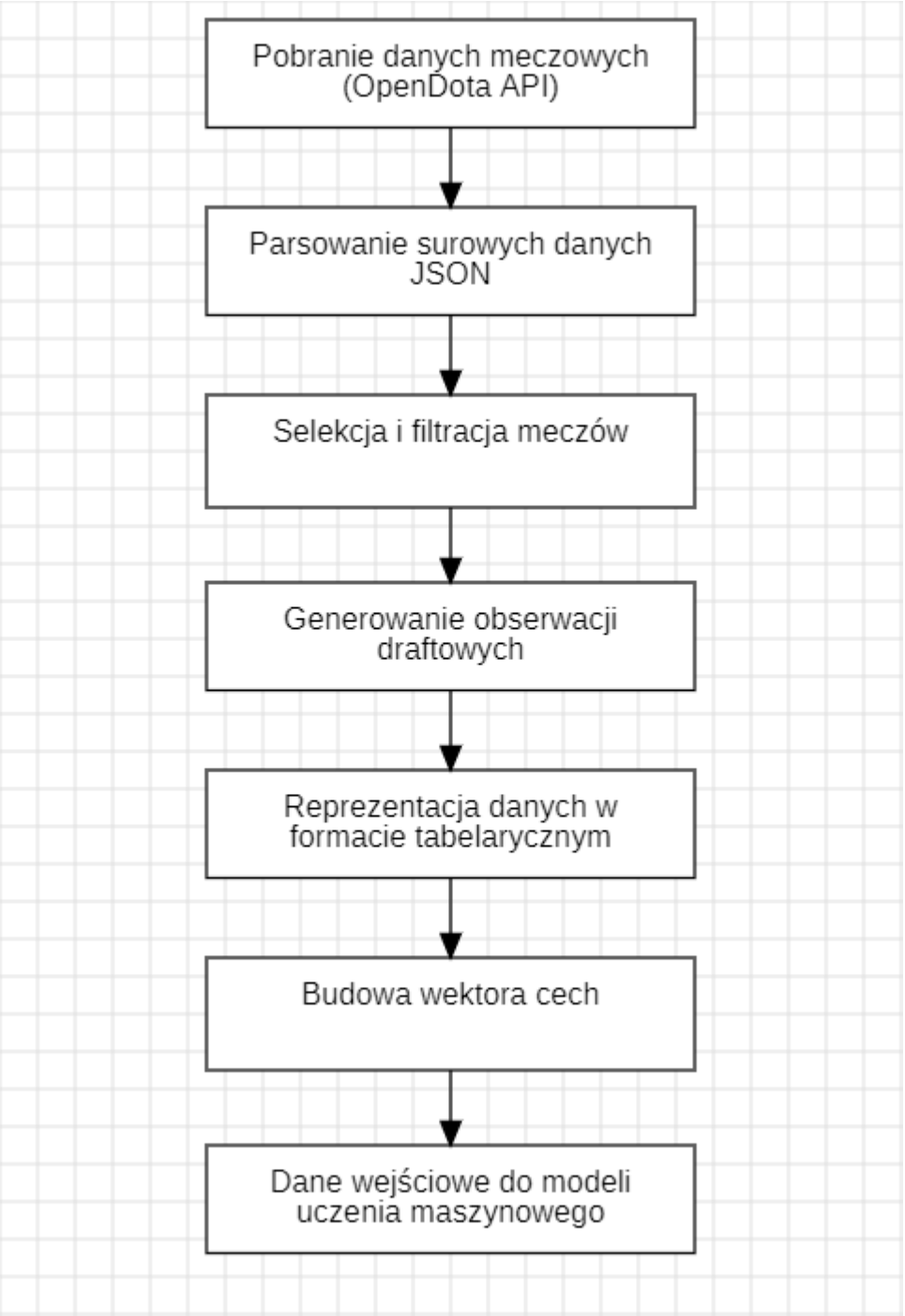
Po połączeniu cech binarnych opisujących obecność poszczególnych bohaterów oraz informacji kontekstowych każda obserwacja przyjmuje postać wektora o dużej liczbie wymiarów, z których zdecydowana większość ma wartość zero. Jest to naturalna konsekwencja zastosowania kodowania one-hot, w którym każda możliwa cecha reprezentowana jest jako osobny wymiar przestrzeni cech.

Zastosowana reprezentacja danych umożliwia efektywne przetwarzanie dużych zbiorów obserwacji, łatwą integrację z bibliotekami uczenia maszynowego, takimi jak scikit-learn oraz XGBoost, a także względnie prostą interpretację wpływu poszczególnych cech na decyzje podejmowane przez model. Pomimo wysokiej wymiarowości wektora cech, struktura danych pozostaje czytelna i dobrze dopasowana do klasycznych algorytmów klasyfikacyjnych.

Każdy rekord w zbiorze danych składa się z wektora cech opisującego aktualny stan draftu oraz etykiety klasy w postaci identyfikatora bohatera wybranego w danym momencie. W ten sposób problem rekomendacji bohaterów zostaje sprowadzony do zadania klasyfikacji wieloklasowej, w którym model przewiduje najbardziej prawdopodobny wybór bohatera w określonym kontekście draftowym.

Schemat całego procesu przetwarzania danych, od danych surowych po finalną postać wektora cech wykorzystywanego przez modele uczenia maszynowego, przedstawiono na rysunku 3.

32:614271



Rysunek 3. Schemat procesu przetwarzania danych od danych surowych do postaci wektora cech wykorzystywanego przez modele uczenia maszynowego (opracowanie własne).

33:215171

4.1.5. Ograniczenia przyjętej reprezentacji

Przyjęta reprezentacja danych, mimo swojej prostoty i skuteczności, posiada pewne ograniczenia. Przede wszystkim nie uwzględnia ona bezpośrednio relacji pomiędzy bohaterami, takich jak synergie czy kontry, a jedynie ich obecność w danym kontekście. Informacje te muszą zostać pośrednio wyuczone przez model na podstawie danych historycznych.

Ponadto reprezentacja nie koduje informacji o rolach bohaterów, ich umiejętnościach ani stylach gry, co może ograniczać zdolność modelu do generalizacji w przypadku zmian w mecie gry.

Z drugiej strony takie uproszczenie pozwoliło na stworzenie systemu o stosunkowo niewielkich wymaganiach obliczeniowych, łatwego do trenowania oraz interpretacji. W kontekście niniejszej pracy przyjęte rozwiązanie stanowi rozsądny kompromis pomiędzy złożonością modelu a dostępnością danych.

4.2. Model bazowy – regresja logistyczna

W procesie projektowania systemu rekomendacyjnego istotne było zastosowanie modelu bazowego, który umożliwiał wstępną ocenę jakości danych oraz sprawdzenie, czy sama reprezentacja cech niesie wystarczającą ilość informacji do przewidywania decyzji draftowych. W tym celu wykorzystano regresję logistyczną jako prosty, klasyczny algorytm klasyfikacyjny.

Regresja logistyczna została wybrana nie jako docelowe rozwiązanie, lecz jako punkt odniesienia dla bardziej złożonych metod. Pozwoliła ona na szybkie przeprowadzenie eksperymentów oraz ocenę, czy problem jest w ogóle rozwiązywalny przy użyciu zaprojektowanej reprezentacji danych.

4.2.1. Uzasadnienie wyboru modelu bazowego

Jednym z głównych powodów zastosowania regresji logistycznej była jej prostota oraz wysoka interpretowalność. Model ten jest często wykorzystywany jako punkt startowy w zadaniach klasyfikacyjnych, ponieważ umożliwia szybkie sprawdzenie, czy w danych istnieją jakiekolwiek zależności pomiędzy cechami a etykietą.

W kontekście niniejszego projektu regresja logistyczna pozwalała odpowiedzieć na pytania:

- czy na podstawie samego stanu draftu można przewidywać wybory bohaterów,
- czy zaprojektowane cechy są wystarczająco informatywne,
- jak duży jest wpływ nierównomiernego rozkładu klas na skuteczność predykcji.

Dodatkową zaletą tego modelu są niskie wymagania obliczeniowe, co umożliwia szybkie trenowanie nawet na dużych zbiorach danych.

4.2.2. Zasada działania

Regresja logistyczna należy do grupy modeli liniowych. W uproszczeniu oznacza to, że próbuje ona opisać zależność pomiędzy cechami wejściowymi a klasą wyjściową za pomocą liniowej kombinacji tych cech.

W praktyce model przypisuje każdej cesze pewną wagę, która określa jej znaczenie w procesie podejmowania decyzji. Następnie obliczana jest suma ważona wszystkich cech, a wynik przekształcany jest do postaci prawdopodobieństwa.

W przypadku klasyfikacji wieloklasowej, z którą mamy do czynienia w tym projekcie, regresja logistyczna przewiduje prawdopodobieństwo przynależności do każdej z możliwych klas, czyli każdego dostępnego bohatera. Ostateczna decyzja podejmowana jest na podstawie najwyższego uzyskanego prawdopodobieństwa.

Model ten nie potrafi jednak w naturalny sposób uchwycić złożonych, nieliniowych zależności pomiędzy cechami, co stanowi jego główne ograniczenie.

4.2.3. Zalety i ograniczenia

Zastosowanie regresji logistycznej wiąże się z szeregiem korzyści. Przede wszystkim model ten jest łatwy w implementacji oraz szybki w trenowaniu. Umożliwia również analizę wag przypisanych poszczególnym cechom, co pozwala na częściową interpretację procesu decyzyjnego.

Jednocześnie regresja logistyczna posiada istotne ograniczenia. Jej liniowy charakter sprawia, że nie radzi sobie dobrze z bardziej złożonymi zależnościami pomiędzy cechami, takimi jak interakcje pomiędzy bohaterami, synergie czy kontry. W przypadku gry Dota 2 takie zależności odgrywają kluczową rolę, co powoduje, że model ten nie jest wystarczający jako główne rozwiązanie.

Z tego względu regresja logistyczna pełni w projekcie rolę modelu referencyjnego, a nie docelowego.

4.2.4. Implementacja w projekcie

W ramach projektu regresja logistyczna została zaimplementowana przy użyciu biblioteki scikit-learn. Model trenowany był na tym samym zbiorze danych oraz przy wykorzystaniu tej samej reprezentacji cech co model docelowy.

Pozwoliło to na bezpośrednie porównanie wyników obu podejść i ocenę, czy zastosowanie bardziej zaawansowanych algorytmów przynosi realne korzyści.

Uzyskane wyniki potwierdziły, że regresja logistyczna jest w stanie uchwycić pewne ogólne tendencje w danych, jednak jej skuteczność była znacząco niższa w porównaniu do modelu XGBoost, który zostanie omówiony w kolejnym podrozdziale.

4.3. Model docelowy – XGBoost

Głównym modelem wykorzystanym w niniejszym projekcie jest algorytm XGBoost (eXtreme Gradient Boosting), który należy do rodziny metod opartych na wzmacnianiu gradientowym drzew decyzyjnych. Został on wybrany ze względu na swoją wysoką skuteczność w zadaniach klasyfikacyjnych, odporność na szum w danych oraz zdolność do modelowania złożonych, nieliniowych zależności pomiędzy cechami.

W przeciwieństwie do regresji logistycznej, która opiera się na liniowej kombinacji cech, XGBoost wykorzystuje zespoły drzew decyzyjnych, co pozwala mu uchwycić skomplikowane interakcje pomiędzy zmiennymi wejściowymi. Ma to kluczowe znaczenie w kontekście gry Dota 2, gdzie decyzje draftowe zależą od subtelnych relacji pomiędzy bohaterami, ich rolami oraz aktualnym kontekstem rozgrywki.

4.3.1. Zasada działania algorytmu gradient boosting

Algorytmy typu gradient boosting opierają się na idei budowy zespołu (ensemble) wielu słabych modeli, które wspólnie tworzą jeden silny model predykcyjny. W praktyce oznacza to, że zamiast trenować jeden złożony model, trenuje się sekwencję prostszych modeli, z których każdy kolejny stara się skorygować błędy poprzednich.

36:786570

W przypadku XGBoost rolę słabych modeli pełnią drzewa decyzyjne. Każde kolejne drzewo jest trenowane w taki sposób, aby minimalizować błąd popełniony przez dotychczasowy zespół modeli. Proces ten jest iteracyjny i prowadzi do stopniowego poprawiania jakości predykcji.

Takie podejście pozwala na efektywne modelowanie nieliniowych zależności oraz interakcji pomiędzy cechami, co jest szczególnie istotne w analizowanym problemie.

4.3.2. Dlaczego XGBoost nadaje się do tego problemu

Problem rekomendacji bohaterów w fazie draftu charakteryzuje się dużą złożonością. Na decyzję wpływają zarówno pojedyncze cechy, jak i ich wzajemne kombinacje. Przykładowo, obecność danego bohatera w drużynie może być korzystna w jednym kontekście, a niekorzystna w innym, w zależności od składu przeciwnika oraz momentu draftu.

XGBoost jest w stanie uchwycić tego typu zależności, ponieważ:

- potrafi modelować relacje nieliniowe,
- automatycznie wykrywa interakcje pomiędzy cechami,
- dobrze radzi sobie z danymi o dużej liczbie wymiarów,
- jest odporny na szum oraz brak idealnej równowagi klas.

Dodatkowym atutem jest jego popularność w środowisku naukowym i przemysłowym, co oznacza dostępność licznych narzędzi do strojenia parametrów oraz analizowania wyników.

4.3.3. Konfiguracja modelu

XGBoost oferuje szeroki zestaw hiperparametrów, które wpływają na zachowanie modelu. W projekcie szczególną uwagę zwrócono na parametry kontrolujące złożoność modelu oraz jego zdolność do generalizacji.

Do najważniejszych należały:

- głębokość drzew decyzyjnych, która wpływa na poziom szczegółowości modelowanych zależności,
- współczynnik uczenia (learning rate), regulujący tempo aktualizacji modelu,
- liczba drzew w zespole,
- parametry regularyzacyjne ograniczające przeuczenie.

Dobór wartości tych parametrów miał na celu uzyskanie kompromisu pomiędzy jakością predykcji a odpornością na przeuczenie.

37:974391

4.3.4. Obsługa klasyfikacji wieloklasowej

Zastosowany problem ma charakter klasyfikacji wieloklasowej, ponieważ model musi wybierać spośród wielu możliwych bohaterów. XGBoost obsługuje taki scenariusz poprzez odpowiednią funkcję celu, która pozwala na jednoczesne przewidywanie prawdopodobieństw przynależności do wszystkich klas.

W praktyce model zwraca wektor prawdopodobieństw, gdzie każda wartość odpowiada jednej klasie. Na tej podstawie generowane są rekomendacje, polegające na wyborze bohaterów o najwyższym przewidywanym prawdopodobieństwie.

Takie podejście umożliwia nie tylko wskazanie jednej najlepszej opcji, lecz również zaproponowanie kilku alternatywnych wyborów, co jest szczególnie użyteczne w kontekście systemu wspomagania decyzji.

4.3.5. Rola modelu w systemie rekomendacyjnym

XGBoost stanowi centralny element całego systemu. To on odpowiada za analizę stanu draftu oraz generowanie rankingów potencjalnych wyborów bohaterów.

W odróżnieniu od prostych systemów regułowych, model nie opiera się na ręcznie zdefiniowanych zasadach, lecz na wzorcach wyuczonych z danych historycznych. Dzięki temu potrafi uwzględniać złożone zależności, które nie są oczywiste dla człowieka.

W kolejnych podrozdziałach przedstawiono sposób trenowania modelu, metody jego ewaluacji oraz proces integracji z aplikacją użytkową.

4.4. Trenowanie modeli

Proces trenowania modeli stanowił kluczowy etap realizacji projektu, ponieważ to na jego podstawie system uczył się zależności pomiędzy stanem draftu a podejmowanymi decyzjami. Celem tego etapu było nie tylko uzyskanie jak najwyższej skuteczności predykcji, lecz również zapewnienie odpowiedniej zdolności modelu do generalizacji na dane niewidziane wcześniej.

Trenowanie modeli zostało przeprowadzone w oparciu o wcześniej przygotowany i przefiltrowany zbiór danych, w którym każda obserwacja odpowiadała pojedynczej decyzji wyboru bohatera wraz z pełnym kontekstem draftu.

38:308943

4.4.1. Podział danych na zbiory treningowe i testowe

Aby umożliwić rzetelną ocenę skuteczności modeli, zbiór danych został podzielony na dwie części: zbiór treningowy oraz zbiór testowy. Zbiór treningowy służył do uczenia parametrów modeli, natomiast zbiór testowy był wykorzystywany wyłącznie do ewaluacji jakości predykcji.

Podział został wykonany w sposób losowy, z zachowaniem proporcji klas (stratyfikacja), co pozwoliło uniknąć sytuacji, w której niektóre klasy byłyby nadreprezentowane w jednej z części. Takie podejście zapewnia bardziej miarodajną ocenę skuteczności modeli w warunkach zbliżonych do rzeczywistych.

4.4.2. Przygotowanie danych wejściowych

Przed rozpoczęciem treningu dane wejściowe zostały przekształcone do postaci macierzy cech, zgodnie z reprezentacją opisaną w poprzednim podrozdziale. Każda obserwacja przyjmowała postać wektora o dużej liczbie wymiarów, zdominowanego przez wartości zerowe, co wynika z zastosowania kodowania one-hot.

Do konwersji słownikowej reprezentacji cech na postać macierzową wykorzystano narzędzia dostępne w bibliotece scikit-learn. Pozwoliło to na automatyczne mapowanie nazw cech na indeksy oraz uzyskanie spójnej struktury danych dla wszystkich obserwacji.

4.4.3. Proces uczenia modelu bazowego

Regresja logistyczna została wykorzystana jako punkt odniesienia dla bardziej złożonych modeli. Jej trening polegał na optymalizacji wag przypisanych do poszczególnych cech w taki sposób, aby maksymalizować prawdopodobieństwo poprawnej klasyfikacji.

Model ten pozwolił na szybkie sprawdzenie poprawności przygotowanej reprezentacji danych oraz wykrycie potencjalnych błędów w pipeline’ie przetwarzania. Dzięki swojej prostocie umożliwił również łatwą interpretację wpływu poszczególnych cech na decyzje predykcyjne.

4.4.4. Proces uczenia modelu XGBoost

Model XGBoost był trenowany w sposób iteracyjny, polegający na stopniowym dodawaniu kolejnych drzew decyzyjnych do zespołu. Każde kolejne drzewo starało się skorygować błędy popełnione przez dotychczasowy model.

39:532241

W trakcie uczenia wykorzystywano mechanizm wczesnego zatrzymania (early stopping), który polega na przerwaniu treningu w momencie, gdy jakość predykcji na zbiorze walidacyjnym przestaje się poprawiać. Pozwoliło to ograniczyć ryzyko przeuczenia oraz skrócić czas trenowania.

Dobór hiperparametrów był realizowany eksperymentalnie, poprzez obserwację wpływu ich zmian na skuteczność predykcji oraz stabilność procesu uczenia. W szczególności testowano wartości głębokości drzew w zakresie od 4 do 8 oraz współczynnik uczenia (learning rate) w przedziale 0,05–0,2, wybierając konfigurację zapewniającą najlepszy kompromis pomiędzy skutecznością predykcji a stabilnością uczenia.

4.4.5. Problemy napotkane w trakcie trenowania

W trakcie realizacji projektu napotkano szereg problemów praktycznych, które miały istotny wpływ na jakość uzyskiwanych predykcji. Jednym z najbardziej zauważalnych zjawisk była nadreprezentacja wybranych bohaterów w wynikach rekomendacji, prowadząca do sytuacji, w której model często sugerował te same postacie niezależnie od kontekstu draftu. Zjawisko to było szczególnie widoczne w przypadku bohaterów popularnych wśród graczy o niższym poziomie zaawansowania, takich jak Pudge.

Początkowo zakładano, że problem ten może zostać rozwiązany poprzez modyfikację architektury modelu lub dostrojenie hiperparametrów algorytmu. Przeprowadzone eksperymenty wykazały jednak, że zmiany po stronie modelu nie przynosiły oczekiwanej poprawy. Analiza wyników wskazała, że główną przyczyną nadmiernej dominacji wybranych bohaterów była charakterystyka danych treningowych, a nie sam proces uczenia.

Znaczna część obserwacji pochodziła z meczów rozgrywanych poza środowiskiem profesjonalnym, w których wybory bohaterów były częściej motywowane indywidualnymi preferencjami graczy niż ugruntowanymi przesłankami strategicznymi. Prowadziło to do sztucznego zawyżenia częstości występowania niektórych postaci, co model interpretował jako silny sygnał decyzyjny.

40:509951

Kluczowym krokiem w rozwiązaniu tego problemu okazało się zaostrzenie kryteriów selekcji danych poprzez podniesienie minimalnego progu średniego poziomu rankingowego (MMR) meczów pobieranych z interfejsu OpenDota API. Ograniczenie zbioru danych do rozgrywek o wysokim poziomie umiejętności sprawiło, że decyzje draftowe stały się bardziej spójne, powtarzalne i oparte na rzeczywistych zależnościach strategicznych charakterystycznych dla aktualnej mety gry. Zmiana ta miała największy wpływ na poprawę jakości rekomendacji i znacząco ograniczyła problem nadmiernej dominacji pojedynczych bohaterów.

Dodatkowym wyzwaniem był silnie nierównomierny rozkład klas, wynikający z faktu, że niektórzy bohaterowie pojawiali się w danych bardzo rzadko. Obecność takich klas prowadziła do niestabilności procesu uczenia oraz obniżenia skuteczności predykcji. W celu poprawy jakości modelu wprowadzono minimalny próg liczby wystąpień bohatera w zbiorze danych. Postacie, które pojawiały się rzadziej niż ustalona liczba razy (40 wyborów w zbiorze liczącym około 3000 meczy), zostały wykluczone z procesu treningowego.

Decyzja ta stanowiła świadomy kompromis pomiędzy różnorodnością klas a skutecznością predykcji. Ograniczenie liczby rzadko wybieranych bohaterów pozwoliło zwiększyć stabilność modelu oraz poprawić jego celność, kosztem zmniejszenia liczby obsługiwanych klas. Zabieg ten nie miał na celu eliminacji problemu nadmiernego faworyzowania popularnych bohaterów, lecz poprawę jakości uczenia poprzez usunięcie klas, dla których dostępna była zbyt mała liczba przykładów treningowych.

Opisane problemy oraz sposoby ich rozwiązania jednoznacznie pokazują, że w projektach opartych na uczeniu maszynowym kluczowe znaczenie ma jakość i odpowiedni dobór danych. W niniejszym projekcie poprawa jakości zbioru treningowego okazała się znacznie skuteczniejsza niż dalsze zwiększanie złożoności modelu.

4.4.6. Podsumowanie etapu trenowania

Proces trenowania modeli umożliwił wyuczenie zależności pomiędzy stanem draftu a decyzjami podejmowanymi przez graczy na wysokim poziomie zaawansowania. Porównanie modelu bazowego oraz modelu XGBoost pozwoliło na ocenę wpływu złożoności modelu na jakość predykcji.

Uzyskane modele stanowiły podstawę do dalszego etapu projektu, którym była ich szczegółowa ewaluacja oraz integracja z aplikacją użytkową.

41:788593

4.5. Ewaluacja modeli i analiza wyników

Opracowanie modelu predykcyjnego nie kończy się na jego wytrenowaniu. Kluczowym etapem jest rzetelna ewaluacja, która pozwala określić, w jakim stopniu model spełnia założone cele projektowe oraz czy generowane przez niego rekomendacje mają rzeczywistą wartość użytkową.

W przypadku niniejszego projektu ewaluacja miała szczególne znaczenie ze względu na specyfikę problemu. Zadanie polega na predykcji kolejnego wyboru bohatera w kontekście dynamicznie zmieniającego się draftu, co oznacza, że poprawność predykcji należy rozpatrywać nie tylko w kategoriach klasycznej trafności, ale również w kontekście użyteczności zaproponowanych alternatyw.

W ramach tego rozdziału opisano zastosowane metryki oceny jakości modeli, procedurę testowania, a także uzyskane wyniki dla poszczególnych podejść. Szczególną uwagę poświęcono porównaniu modelu bazowego z modelem XGBoost oraz analizie wpływu zastosowanych metod filtracji danych i ograniczeń zbioru klas. Na końcu przedstawiono interpretację wyników w kontekście rzeczywistych decyzji podejmowanych przez graczy oraz omówiono ograniczenia przeprowadzonej ewaluacji.

4.5.1. Metryki oceny jakości predykcji

Ocena jakości działania modelu predykcyjnego wymaga zastosowania odpowiednich metryk, które w sposób ilościowy opisują stopień dopasowania modelu do danych testowych. W klasycznych zadaniach klasyfikacyjnych często stosuje się miary takie jak dokładność (accuracy), precyzja, czułość czy miara F1. Jednak w przypadku niniejszego projektu takie podejście okazało się niewystarczające.

Zadanie polega na predykcji kolejnego wyboru bohatera spośród bardzo dużej liczby możliwych klas, a sam system ma charakter rekomendacyjny. Oznacza to, że w praktyce użytkownik nie oczekuje jednej „idealnej” odpowiedzi, lecz zestawu sensownych propozycji, spośród których może wybrać najbardziej odpowiednią. Z tego względu zastosowano metryki uwzględniające ranking predykcji, a nie wyłącznie pojedynczą klasę o najwyższym prawdopodobieństwie.

Podstawową miarą zastosowaną w projekcie była metryka Top-k Accuracy. Określa ona, czy rzeczywisty wybór bohatera znajduje się wśród pierwszych k propozycji zwróconych przez model. Przykładowo, dla wartości k = 5, predykcja uznawana jest za poprawną, jeśli prawdziwa klasa znajduje się wśród pięciu najwyżej ocenionych przez model bohaterów.

Zastosowanie tej metryki jest szczególnie uzasadnione w kontekście fazy draftu, gdzie istnieje wiele poprawnych decyzji strategicznych, a wybór jednej z nich często zależy od indywidualnych preferencji gracza, stylu drużyny czy aktualnej sytuacji turniejowej. W takim ujęciu system nie musi wskazywać dokładnie tego samego bohatera, który został wybrany w rzeczywistości, aby jego rekomendacje miały wartość użytkową.

4.5.2. Podział danych na zbiory treningowe i testowe

Aby rzetelnie ocenić zdolność modelu do generalizacji, dane zostały podzielone na zbiór treningowy oraz zbiór testowy. Zbiór treningowy służył do uczenia parametrów modeli, natomiast zbiór testowy był wykorzystywany wyłącznie do oceny ich skuteczności.

Podział danych został przeprowadzony w sposób losowy z zachowaniem proporcji klas (stratyfikacja). Było to szczególnie istotne ze względu na silnie niezrównoważony rozkład klas, niektórzy bohaterowie pojawiali się w danych znacznie częściej niż inni. Brak stratyfikacji mógłby prowadzić do sytuacji, w której rzadziej występujące klasy nie byłyby w ogóle reprezentowane w zbiorze testowym.

Dodatkowo zadbano o to, aby do zbioru testowego nie trafiały obserwacje, które w sposób pośredni mogłyby ujawniać informacje o przyszłych decyzjach w tym samym meczu. Pozwoliło to uniknąć zjawiska tzw. przecieku informacji (data leakage), który mógłby prowadzić do sztucznie zawyżonych wyników.

4.5.3. Porównanie modeli bazowych i zaawansowanych

W ramach projektu zastosowano dwa główne podejścia modelowe: regresję logistyczną jako model bazowy oraz XGBoost jako model bardziej zaawansowany.

Regresja logistyczna pełniła rolę punktu odniesienia. Jej prostota oraz liniowy charakter pozwalały na szybkie sprawdzenie, czy przyjęta reprezentacja danych zawiera wystarczającą ilość informacji, aby możliwe było przewidywanie decyzji draftowych.

Model XGBoost umożliwiał natomiast modelowanie nieliniowych zależności oraz interakcji pomiędzy cechami. Dzięki wykorzystaniu zespołów drzew decyzyjnych był on w stanie lepiej uchwycić złożone relacje pomiędzy zestawami bohaterów, momentem draftu oraz podejmowaną decyzją.

Porównanie obu modeli pozwoliło na ocenę, w jakim stopniu zwiększenie złożoności algorytmu przekłada się na realną poprawę jakości rekomendacji.

4.5.4. Analiza błędów i ograniczeń predykcji

Oprócz ilościowej oceny jakości modeli przeprowadzono również analizę jakościową generowanych rekomendacji. Pozwoliło to na identyfikację przypadków, w których model generował predykcje nieintuicyjne lub odbiegające od rzeczywistych decyzji graczy.

Szczególną uwagę zwrócono na sytuacje, w których model wielokrotnie rekomendował tego samego bohatera niezależnie od kontekstu draftu. Zjawisko to było często wynikiem silnej nierównowagi klas oraz dominacji niektórych postaci w danych historycznych.

Zaobserwowano również przypadki, w których model ignorował subtelne zależności kontekstowe, takie jak synergie pomiędzy konkretnymi bohaterami, jeśli nie były one wystarczająco często reprezentowane w danych treningowych.

Tego typu analiza była kluczowa dla zrozumienia nie tylko tego, jak dobrze model działa, ale również dlaczego popełnia określone błędy.

4.5.5. Wnioski z procesu ewaluacji

Przeprowadzona ewaluacja wykazała, że zastosowane modele są w stanie uchwycić istotne zależności pomiędzy kontekstem draftu a podejmowanymi decyzjami. Szczególnie widoczna była przewaga modelu XGBoost nad regresją logistyczną, co potwierdza zasadność zastosowania bardziej złożonych algorytmów.

W przeprowadzonych eksperymentach model XGBoost osiągnął dokładność top-1 accuracy na poziomie około 18%, podczas gdy regresja logistyczna uzyskała wynik rzędu 12%. Różnica ta była jeszcze bardziej widoczna w przypadku metryki top-5 accuracy, gdzie model XGBoost poprawił wynik o około 10 punktów procentowych w porównaniu do modelu bazowego.

44:620616

Jednocześnie analiza wyników uwidoczniła ograniczenia wynikające z charakteru danych historycznych oraz przyjętej reprezentacji cech. Modele w dużej mierze odzwierciedlają dominujące trendy obecne w danych, co oznacza, że ich skuteczność może ulegać pogorszeniu wraz ze zmianami mety gry.

Uzyskane rezultaty potwierdziły, że system może pełnić rolę narzędzia wspomagającego decyzje graczy, jednak nie powinien być traktowany jako źródło jednoznacznych i nieomylnych rekomendacji.

5. Rozdział 5 – Implementacja aplikacji i interfejs użytkownika

W poprzednich rozdziałach przedstawiono proces projektowania systemu, przygotowania danych oraz budowy i ewaluacji modeli uczenia maszynowego. Sam model predykcyjny nie stanowi jednak kompletnego rozwiązania — aby mógł on zostać wykorzystany w praktyce, konieczne jest stworzenie odpowiedniej aplikacji umożliwiającej interakcję użytkownika z systemem.

Celem niniejszego rozdziału jest opisanie procesu implementacji kompletnego systemu rekomendacyjnego, obejmującego zarówno warstwę backendową odpowiedzialną za logikę działania i komunikację z modelem, jak i warstwę frontendową, czyli interfejs użytkownika. Szczególny nacisk położono na prostotę obsługi, czytelność prezentowanych informacji oraz intuicyjność interakcji.

Zaprojektowana aplikacja umożliwia użytkownikowi podanie identyfikatora meczu oraz nazwy bohatera, którym grał, a następnie generuje rekomendacje alternatywnych wyborów na podstawie wytrenowanego modelu. System działa w trybie post-game, odtwarzając rzeczywisty kontekst draftu na podstawie danych pobranych z OpenDota API.

W dalszej części rozdziału omówiono architekturę aplikacji, sposób integracji modelu z backendem, strukturę interfejsu użytkownika oraz napotkane problemy implementacyjne.

5.1. Architektura systemu

Zaprojektowany system składa się z kilku niezależnych komponentów, które wspólnie tworzą spójną aplikację umożliwiającą generowanie rekomendacji draftowych. Architektura została zaprojektowana w sposób modułowy, co ułatwia rozwój, testowanie oraz ewentualną rozbudowę systemu w przyszłości.

45:121624

Podstawowy podział obejmuje trzy główne warstwy: warstwę danych, warstwę logiki aplikacyjnej oraz warstwę prezentacji. Warstwa danych odpowiada za komunikację z zewnętrznym API, pobieranie informacji o meczach oraz ich wstępne przetwarzanie. Warstwa logiki aplikacyjnej zawiera implementację mechanizmów budowy cech, wywoływania modelu oraz generowania rekomendacji. Warstwa prezentacji obejmuje interfejs użytkownika umożliwiający wprowadzanie danych oraz wizualizację wyników.

W ramach realizacji projektu zastosowano język Python jako główne środowisko programistyczne, framework FastAPI do budowy backendu, bibliotekę XGBoost jako silnik predykcyjny oraz Streamlit jako narzędzie do budowy interfejsu użytkownika. Dane meczowe pobierane są dynamicznie z OpenDota API.

Takie podejście pozwala na wyraźne oddzielenie odpowiedzialności poszczególnych komponentów i zwiększa czytelność całego projektu.

5.2. Implementacja backendu

Warstwa backendowa systemu została zaimplementowana przy użyciu frameworka FastAPI, który umożliwia szybkie tworzenie wydajnych i czytelnych interfejsów programistycznych REST. Backend pełni rolę pośrednika pomiędzy interfejsem użytkownika a modelem uczenia maszynowego.

Do jego zadań należy obsługa zapytań wysyłanych przez użytkownika, pobieranie danych meczowych z interfejsu OpenDota API, rekonstrukcja kontekstu fazy draftu oraz przygotowanie danych wejściowych w postaci wektora cech. Następnie backend przekazuje przygotowane dane do modelu predykcyjnego i odpowiada za zwrócenie wyników w ustrukturyzowanej postaci.

Zastosowanie FastAPI umożliwiło wykorzystanie automatycznej walidacji danych wejściowych, generowanie dokumentacji API oraz prostą obsługę błędów, co znacząco ułatwiło rozwój i testowanie systemu.

46:963318

5.3. Endpoint rekomendacyjny

Centralnym elementem backendu jest endpoint odpowiedzialny za generowanie rekomendacji. Przyjmuje on jako dane wejściowe identyfikator meczu oraz nazwę bohatera, którym grał użytkownik.

Po odebraniu żądania system wykonuje następujące kroki:

1. Normalizuje nazwę bohatera i mapuje ją na identyfikator wewnętrzny.
2. Pobiera dane meczu z OpenDota API.
3. Odtwarza sekwencję zdarzeń draftowych.
4. Wyszukuje moment, w którym użytkownik dokonał wyboru.
5. Rekonstruuje stan draftu na ten moment.
6. Buduje wektor cech.
7. Przekazuje dane do modelu.
8. Odbiera predykcję i filtruje bohaterów już wybranych lub zablokowanych.
9. Zwraca listę rekomendacji.

Takie podejście pozwala na realistyczne odtworzenie warunków, w jakich decyzja była podejmowana.

5.4. Integracja modelu z aplikacją

Model uczenia maszynowego został zapisany na dysku w postaci plików binarnych i ładowany do pamięci podczas startu aplikacji. Dzięki temu każda predykcja może być wykonywana bez konieczności ponownego trenowania modelu.

Zastosowano również wektoryzator cech oraz enkoder etykiet, które muszą być zgodne z konfiguracją używaną podczas treningu. Wszystkie te elementy są inicjalizowane jednorazowo.

Podczas generowania rekomendacji model zwraca rozkład prawdopodobieństw dla wszystkich klas. Następnie system sortuje wyniki malejąco, usuwa bohaterów niedostępnych w danym momencie draftu i wybiera pierwsze K propozycji.

47:804821

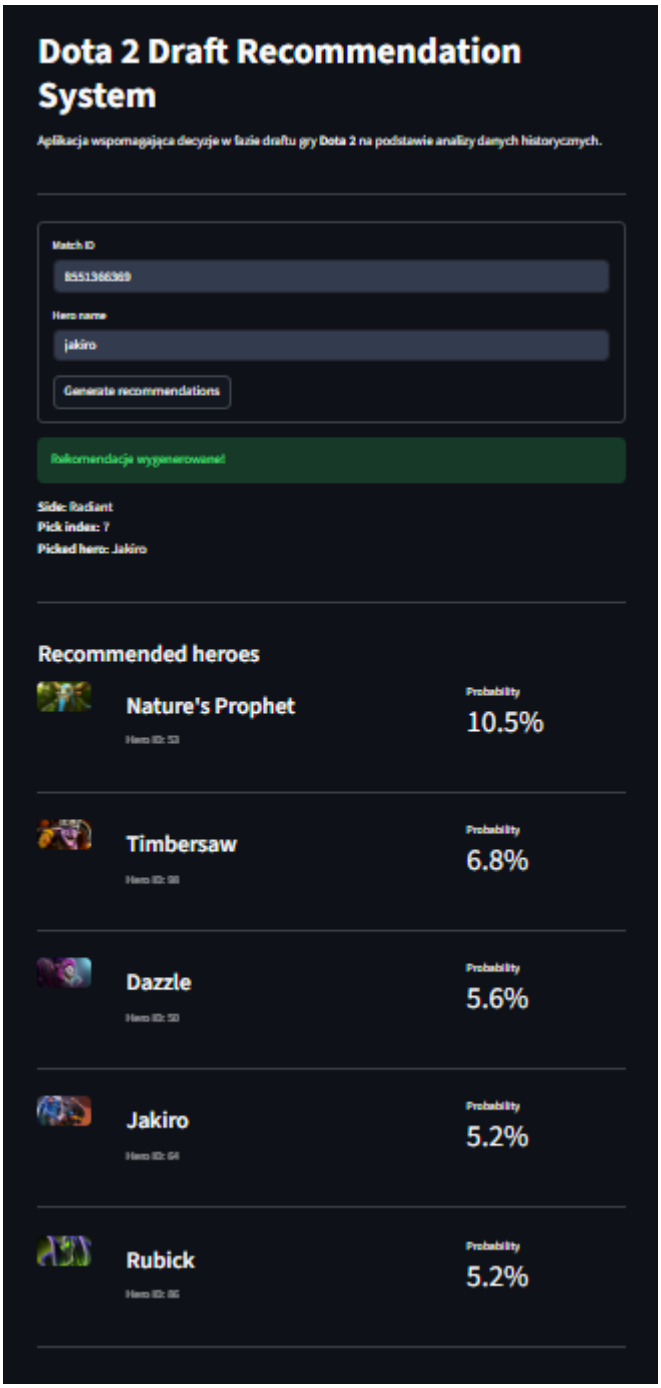
5.5. Implementacja interfejsu użytkownika

Warstwa frontendowa została zaimplementowana przy użyciu biblioteki Streamlit, która umożliwia szybkie tworzenie aplikacji webowych bez konieczności pisania kodu HTML, CSS czy JavaScript.

Interfejs użytkownika składa się z prostego formularza umożliwiającego wprowadzenie identyfikatora meczu oraz nazwy bohatera. Po wysłaniu formularza aplikacja komunikuje się z backendem, a następnie prezentuje wyniki w czytelnej formie.

Wyniki wyświetlane są jako lista rekomendowanych bohaterów wraz z ich ikonami oraz prawdopodobieństwami w postaci procentowej. Zastosowanie wizualnych elementów znacząco zwiększa czytelność oraz atrakcyjność interfejsu, co ułatwia użytkownikowi analizę proponowanych rekomendacji.

Przykładowy widok interfejsu użytkownika aplikacji, prezentujący formularz wejściowy oraz wyniki rekomendacji, przedstawiono na rysunku 4.



Rysunek 4. Widok interfejsu użytkownika aplikacji rekomendacyjnej zaimplementowanej w technologii Streamlit (opracowanie własne).

49:178610

5.6. Problemy napotkane podczas implementacji

W trakcie realizacji projektu napotkano szereg problemów natury technicznej oraz implementacyjnej, wynikających zarówno z integracji wielu komponentów systemu, jak i z ograniczeń zewnętrznych źródeł danych. Problemy te dotyczyły w szczególności komunikacji z interfejsem OpenDota API, budowy interfejsu użytkownika oraz integracji modelu uczenia maszynowego z warstwą aplikacyjną. Ich rozwiązanie miało istotny wpływ na stabilność oraz użyteczność końcowego systemu.

Jednym z pierwszych wyzwań była integracja systemu z OpenDota API. Interfejs ten nie gwarantuje natychmiastowej dostępności pełnych danych dla każdego meczu, ponieważ proces parsowania rozgrywek po ich zakończeniu może trwać od kilku minut do kilku godzin. Prowadziło to do sytuacji, w których użytkownik wprowadzał poprawny identyfikator meczu, lecz system nie był w stanie pobrać informacji o fazie draftu. Problem rozwiązano poprzez dodanie odpowiednich mechanizmów obsługi błędów oraz komunikatów informujących użytkownika o niedostępności danych.

Kolejnym istotnym problemem były przekroczenia czasu oczekiwania (timeouty) podczas pobierania danych z API. W początkowych wersjach aplikacji długie zapytania powodowały zawieszanie się interfejsu użytkownika, co skutkowało brakiem odpowiedzi oraz wrażeniem nieskończonego ładowania. W celu rozwiązania tego problemu wprowadzono limity czasu dla zapytań HTTP oraz odpowiednią obsługę wyjątków, co pozwoliło na zachowanie responsywności aplikacji.

Podczas implementacji interfejsu użytkownika w technologii Streamlit pojawiły się trudności związane z wyświetlaniem wyników w czytelnej i atrakcyjnej formie. Początkowo rekomendacje prezentowane były jedynie jako listy identyfikatorów bohaterów, co znacząco obniżało użyteczność systemu. Aby poprawić doświadczenie użytkownika, zdecydowano się na prezentację nazw bohaterów oraz ich ikon graficznych. Wymagało to stworzenia mechanizmu mapowania identyfikatorów na nazwy oraz dynamicznego generowania adresów URL prowadzących do grafik przechowywanych na serwerach OpenDota.

Proces ten nie był trywialny, ponieważ nazwy bohaterów w API nie zawsze odpowiadały bezpośrednio konwencji stosowanej w adresach URL obrazów. Konieczne było zastosowanie normalizacji nazw, polegającej m.in. na usuwaniu znaków specjalnych, zamianie spacji na znaki podkreślenia oraz ujednoliceniu wielkości liter. Dopiero po wprowadzeniu tych mechanizmów udało się uzyskać poprawne wyświetlanie ikon w interfejsie użytkownika.

50:529819

Dodatkowym wyzwaniem była synchronizacja działania backendu opartego na FastAPI z frontendem stworzonym w Streamlit. W początkowych wersjach aplikacji występowały problemy z wielokrotnym wykonywaniem zapytań oraz zapętlaniem procesu rekomendacji, co skutkowało znacznym obciążeniem serwera oraz długimi czasami odpowiedzi. Problem rozwiązano poprzez wprowadzenie mechanizmów kontrolujących moment wysyłania zapytań oraz odpowiednie zarządzanie stanem aplikacji po stronie interfejsu użytkownika.

Warto również zaznaczyć, że część problemów miała charakter eksperymentalny i wynikała z iteracyjnego charakteru projektu. System był wielokrotnie modyfikowany, a poszczególne komponenty podlegały refaktoryzacji. Takie podejście umożliwiło stopniowe eliminowanie błędów, poprawę struktury kodu oraz zwiększenie jego czytelności.

Podsumowując, napotkane trudności miały istotny wpływ na ostateczny kształt aplikacji. Ich rozwiązanie przyczyniło się nie tylko do poprawy stabilności systemu, lecz również do zwiększenia jego użyteczności z perspektywy końcowego użytkownika. Wiele z tych problemów nie dotyczyło bezpośrednio jakości modelu uczenia maszynowego, lecz miało kluczowe znaczenie dla praktycznej wartości opracowanego rozwiązania.

5.7. Podsumowanie

W niniejszym rozdziale przedstawiono proces implementacji kompletnego systemu rekomendacyjnego, obejmującego zarówno warstwę backendową, jak i frontendową. Zastosowana architektura umożliwia modularny rozwój aplikacji oraz jej łatwą adaptację do przyszłych zmian.

Stworzona aplikacja stanowi praktyczne narzędzie umożliwiające wykorzystanie wytrenowanego modelu w realistycznym scenariuszu użytkowym. Integracja modelu z interfejsem pozwala na ocenę jego działania w warunkach zbliżonych do rzeczywistego użycia.

51:107833

6. Rozdział 6 – Podsumowanie i kierunki dalszego rozwoju

Celem niniejszej pracy było zaprojektowanie oraz implementacja systemu rekomendacyjnego wspomagającego graczy gry Dota 2 w podejmowaniu decyzji podczas fazy draftu. W przeciwieństwie do klasycznych systemów rekomendacyjnych, analizowany problem charakteryzuje się silną zależnością od kontekstu, dużą liczbą możliwych klas oraz sekwencyjnym charakterem procesu decyzyjnego. Każda decyzja wpływa na przestrzeń możliwych kolejnych wyborów, co znacząco zwiększa złożoność zadania.

W ramach projektu opracowano kompletny pipeline obejmujący pozyskiwanie danych, ich filtrację i przetwarzanie, konstrukcję zbioru obserwacji draftowych, budowę reprezentacji cech, trenowanie modeli klasyfikacyjnych oraz stworzenie aplikacji umożliwiającej interakcję użytkownika z systemem. Zastosowane podejście pozwoliło na przekształcenie surowych danych meczowych w uporządkowaną postać umożliwiającą efektywne uczenie modeli.

Szczególną uwagę poświęcono jakości danych, która okazała się jednym z kluczowych czynników wpływających na skuteczność predykcji. W trakcie realizacji projektu wykazano, że nawet zaawansowane modele uczenia maszynowego nie są w stanie generować wartościowych rekomendacji, jeśli dane treningowe zawierają duży udział losowych lub niestandardowych decyzji. Z tego względu wprowadzono mechanizmy selekcji meczów o wysokim poziomie umiejętności oraz dodatkowe kryteria filtracji, które pozwoliły na uzyskanie bardziej spójnego i reprezentatywnego zbioru uczącego.

W projekcie zastosowano algorytm XGBoost jako główny model predykcyjny. Jego wykorzystanie pozwoliło na skuteczne modelowanie zależności pomiędzy cechami opisującymi stan draftu oraz generowanie trafnych rekomendacji w warunkach nierównomiernego rozkładu danych. Zastosowany model spełnił założenia projektowe i umożliwił realizację wszystkich funkcjonalności systemu rekomendacyjnego.

Istotnym elementem projektu była również implementacja aplikacji użytkowej, która umożliwia korzystanie z wytrenowanego modelu w sposób przystępny i intuicyjny. Użytkownik podaje identyfikator meczu oraz nazwę bohatera, a system automatycznie rekonstruuje stan draftu i generuje zestaw alternatywnych rekomendacji. Taka forma interakcji pozwala na analizę decyzji w trybie post-game oraz umożliwia lepsze zrozumienie mechanizmów stojących za podejmowanymi wyborami.

52:760580

6.1. Ograniczenia zaproponowanego rozwiązania

Pomimo uzyskanych rezultatów, zaproponowany system posiada istotne ograniczenia. Przede wszystkim opiera się on wyłącznie na danych historycznych i nie uwzględnia dynamicznych zmian balansu gry. Oznacza to, że w przypadku istotnych aktualizacji mechaniki, bohaterów lub przedmiotów skuteczność rekomendacji może ulec pogorszeniu.

Dodatkowo zastosowana reprezentacja danych nie koduje wprost informacji o rolach bohaterów, ich umiejętnościach ani bezpośrednich relacjach, takich jak synergie czy kontry. Informacje te muszą być pośrednio wyuczone przez model na podstawie wzorców występujących w danych, co może ograniczać jego zdolność do generalizacji w sytuacjach nietypowych.

Kolejnym kompromisem było ograniczenie liczby klas poprzez usunięcie bohaterów pojawiających się bardzo rzadko w danych treningowych. Zabieg ten pozwolił na zwiększenie stabilności uczenia oraz poprawę jakości predykcji, jednak odbyło się to kosztem redukcji liczby dostępnych bohaterów, które mogą zostać zarekomendowane przez system.

6.2. Kierunki dalszego rozwoju

Istnieje wiele możliwości rozbudowy zaproponowanego systemu. Jednym z najbardziej naturalnych kierunków jest implementacja trybu działania w czasie rzeczywistym, w którym rekomendacje generowane byłyby na bieżąco w trakcie trwania draftu. Wymagałoby to integracji z interfejsami umożliwiającymi odczyt aktualnego stanu wyborów oraz dalszej optymalizacji czasu odpowiedzi modelu.

Kolejnym istotnym rozszerzeniem byłoby wprowadzenie modułu rekomendacji przedmiotów. Taki system mógłby uwzględniać nie tylko wybranego bohatera, ale również skład drużyn, przebieg meczu oraz aktualną sytuację na mapie. Pozwoliłoby to na stworzenie bardziej kompleksowego narzędzia wspomagającego decyzje graczy nie tylko na etapie draftu, lecz również w trakcie samej rozgrywki.

Warto również rozważyć implementację mechanizmów ciągłego uczenia się na nowych danych. Dzięki temu system mógłby automatycznie adaptować się do zmian w mecie gry, bez konieczności ręcznego ponownego trenowania modeli po każdej większej aktualizacji.

Dalsze badania mogłyby także objąć zastosowanie bardziej zaawansowanych reprezentacji danych, takich jak modele sekwencyjne lub grafowe, które lepiej oddają zależności pomiędzy kolejnymi decyzjami oraz relacje pomiędzy bohaterami.

53:455803

6.3. Podsumowanie końcowe

Przeprowadzony projekt pokazał, że możliwe jest stworzenie systemu rekomendacyjnego zdolnego do analizy złożonych procesów decyzyjnych w grach typu MOBA przy wykorzystaniu rzeczywistych danych meczowych. Otrzymane wyniki potwierdzają, że odpowiednio przygotowane dane oraz właściwie dobrana reprezentacja cech mają kluczowe znaczenie dla skuteczności modeli uczenia maszynowego.

Zaproponowane rozwiązanie nie ma na celu zastąpienia gracza, lecz stanowi narzędzie wspomagające proces podejmowania decyzji. Może ono służyć zarówno jako pomoc dydaktyczna, jak i narzędzie analityczne umożliwiające refleksję nad własnymi wyborami oraz lepsze zrozumienie mechanizmów rządzących rozgrywką.

54:110661

Bibliografia

KSIĄŻKI

Bengio, Y., Goodfellow, I., Courville, A. (2016). Deep Learning. MIT Press.

Hastie, T., Tibshirani, R., Friedman, J. (2009). The Elements of Statistical Learning. Springer.

James, G., Witten, D., Hastie, T., Tibshirani, R. (2021). An Introduction to Statistical Learning. Springer.

Kelleher, J. D., Namee, B. M., D'Arcy, A. (2020). Fundamentals of Machine Learning for Predictive Data Analytics. MIT Press.

ARTYKUŁY NAUKOWE / CZASOPISMA

Chen, T., Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Koren, Y., Bell, R., Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer Society.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Silver, D. et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science.

55:113113

DOKUMENTACJA TECHNICZNA I NARZĘDZIA

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

Python Software Foundation. (2024). Python Language Reference. https://www.python.org
(dostęp: 10.2025).

Scikit-learn Developers. (2024). Scikit-learn Documentation. https://scikit-learn.org
(dostęp: 10.2025).

XGBoost Developers. (2024). XGBoost Documentation. https://xgboost.readthedocs.io
(dostęp: 10.2025).

FastAPI Developers. (2024). FastAPI Documentation. https://fastapi.tiangolo.com
(dostęp: 10.2025).

Streamlit Developers. (2024). Streamlit Documentation. https://streamlit.io
(dostęp: 10.2025).

ŹRÓDŁA INTERNETOWE

OpenDota. (2024). OpenDota API Documentation.
https://docs.opendota.com
(dostęp: 10.2025).

Valve Corporation. (2024). Dota 2.
https://www.dota2.com
(dostęp: 10.2025).

56:104057

Valve Developer Wiki. (2024). Dota 2 Draft Mechanics.
https://developer.valvesoftware.com
(dostęp: 10.2025).

Kaggle. (2024). Machine Learning Evaluation Metrics.
https://www.kaggle.com
(dostęp: 10.2025).